

Understanding Artificial Intelligence: From Mathematics to Applications

Lecture 9: *Deep(est) learning*

Jean Feng

Logistics

- Memorial Day next week — no class.
- HW4 due next week.
- Project check-ins due next week.
- Final project presentations on 6/1 and 6/8. You will be assigned randomly. You are expected to come attend both lectures (unless you have extenuating circumstances).
 - Come support your fellow students and learn about the variety of methods everyone is exploring!
 - After each presentation, there will be a few minutes for a question from the instructor and a question from a fellow student.
- Final project reports due 6/8.
 - Extra credit for doing peer review.
- And that will be all folks!

Last week

- Representation learning
- Convolutional neural networks (CNNs)
- Residual networks
- Recurrent neural networks (RNNs)
 - LSTMs

Lecture Outline

- **Recall: Encoder-Decoder Network**
- Attention mechanism
- Transformers
- Generative Pretrained Transformers (GPT)

Recall: Recurrent neural networks for predicting the next word

So ____

So long ____

So long and ____

So long and thanks ____

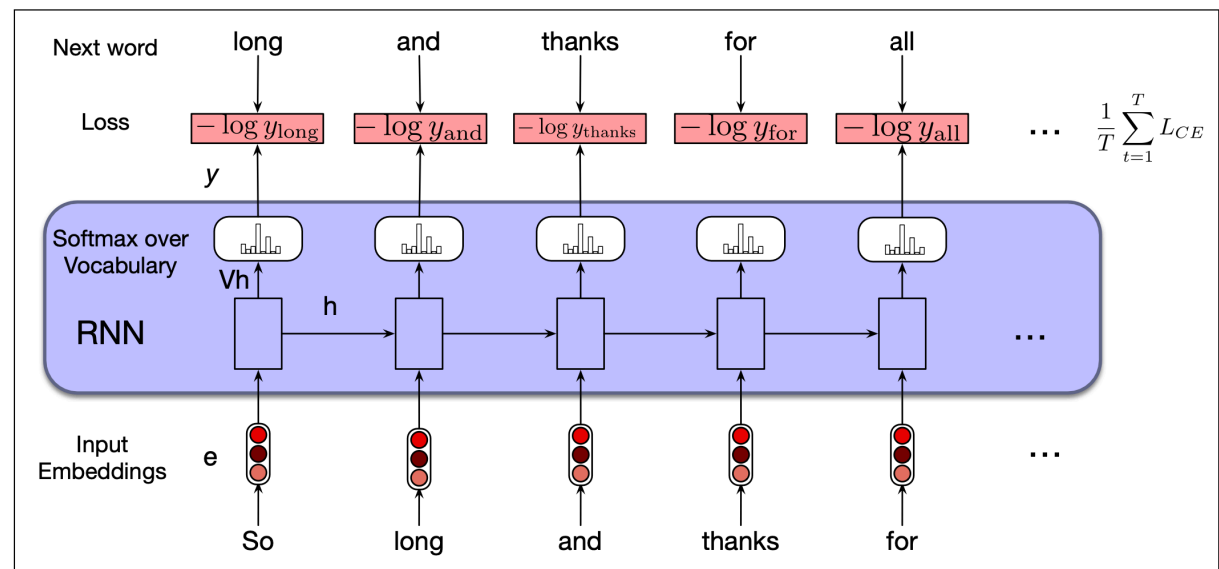
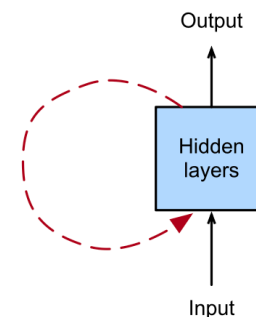


Figure 9.6 Training RNNs as language models.



Quiz

- What's the objective function for training an RNN to predict the next word in a sequence?
- A: Mean squared error
- B: Binary cross-entropy (binary classification)
- C: Cross-entropy (multi-class classification)
- D: None of the above

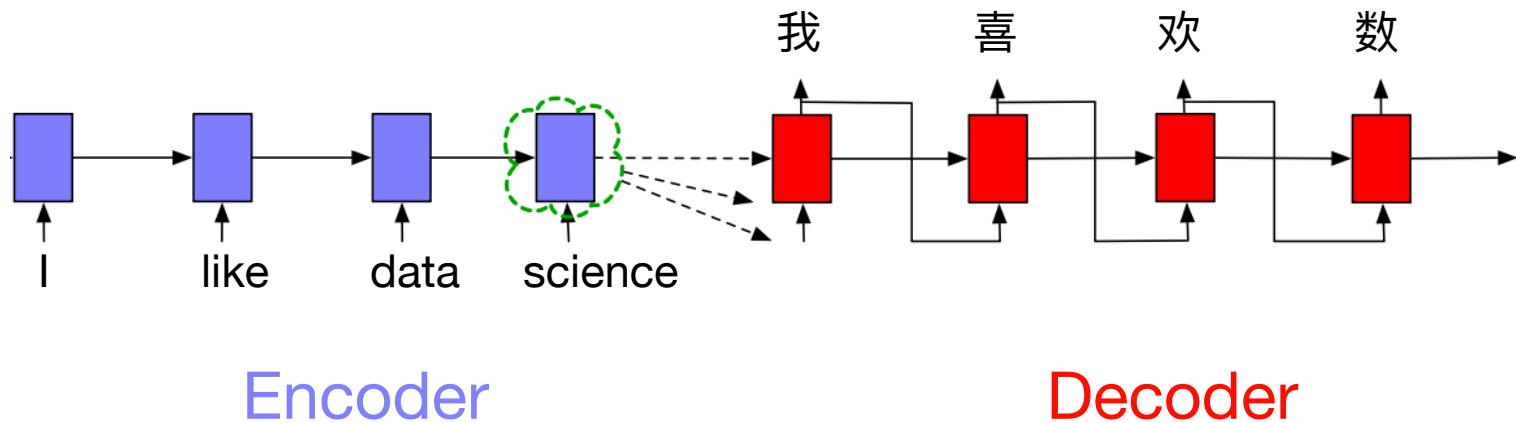
Recurrent neural networks for sequence-to-sequence tasks

Translation task:

I like data science.

我喜欢数据科学。

Encoder-decoder networks



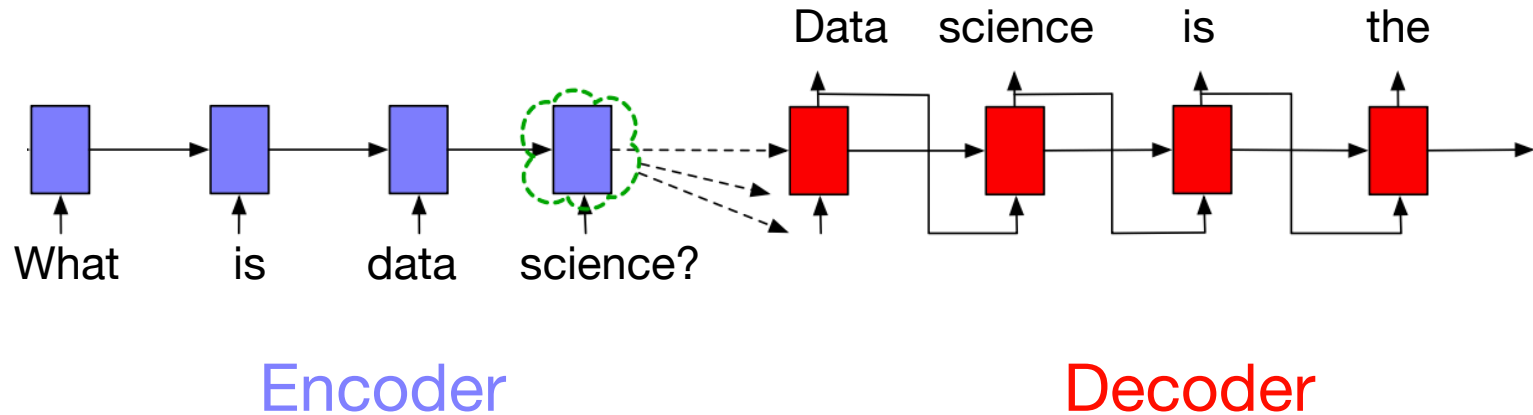
Recurrent neural networks for sequence-to-sequence tasks

QA task:

What is data science?

Data science is the field...

Encoder-decoder networks



Lecture Outline

- Encoder-Decoder Network
- **Attention mechanism**
- Transformers
- Generative Pretrained Transformers (GPT)

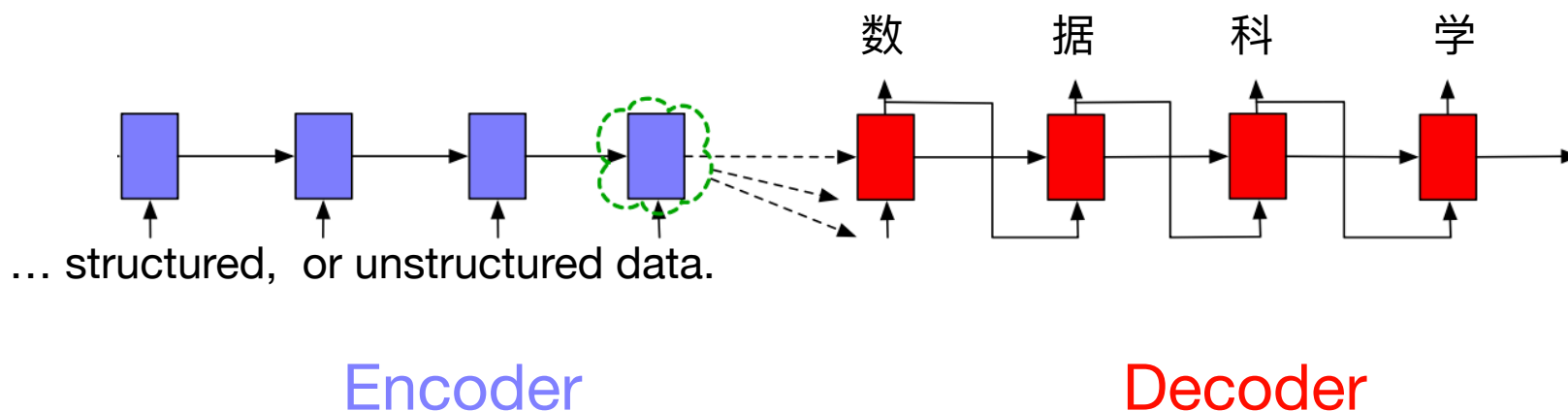
The problem of long inputs

Translation task:

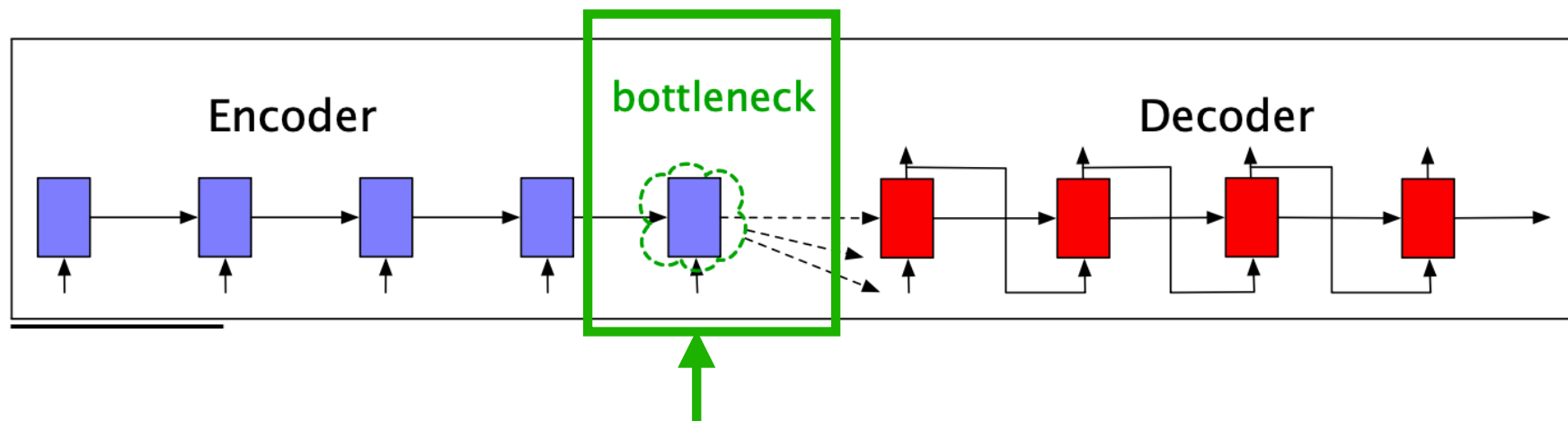
Data science is an interdisciplinary academic field that uses statistics, scientific computing, scientific methods, processes, algorithms and systems to extract or extrapolate knowledge and insights from potentially noisy, structured, or unstructured data.

数据科学是一个跨学科的学术领域，它使用统计学、科学计算、科学方法、流程、算法和系统从潜在的噪声、结构化或非结构化数据中提取或推断知识和见解。

Encoder-decoder networks



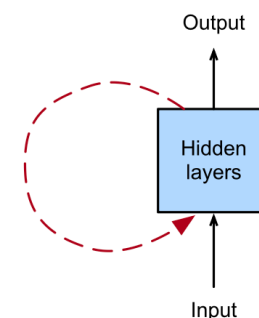
The problem with bottlenecks



A standard RNN/LSTM summarizes the entire input into one fixed-size vector, regardless of how long the input sentence is.

Compression of variable-sized inputs into a fixed-size vector is inevitably lossy, which can hurt prediction.

How can we have a recurrent structure to process variable-sized inputs but have memory that grows with the size of the input?



Fixing the problem of bottlenecks with attention

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau

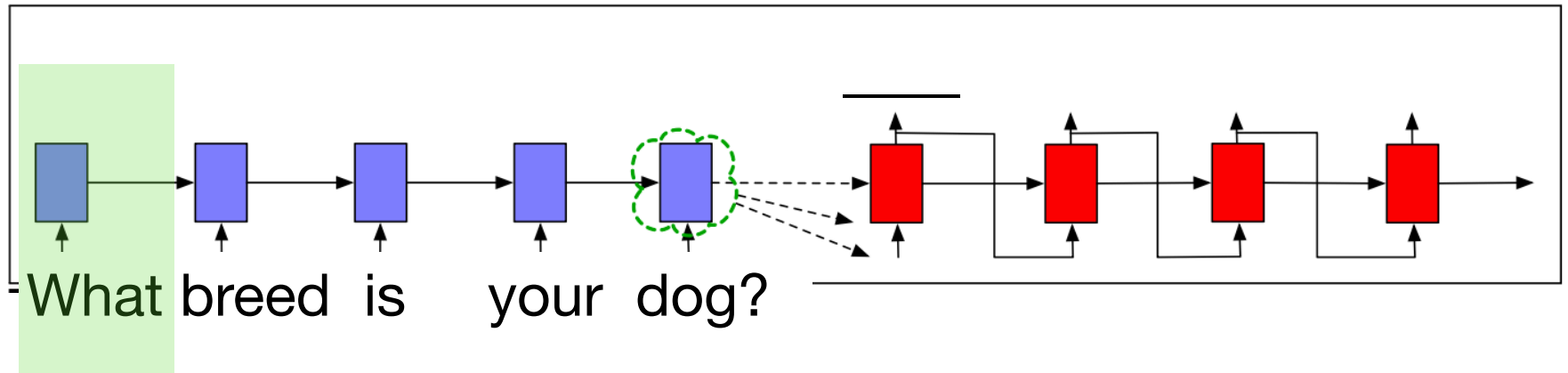
Jacobs University Bremen, Germany

KyungHyun Cho Yoshua Bengio*

Université de Montréal

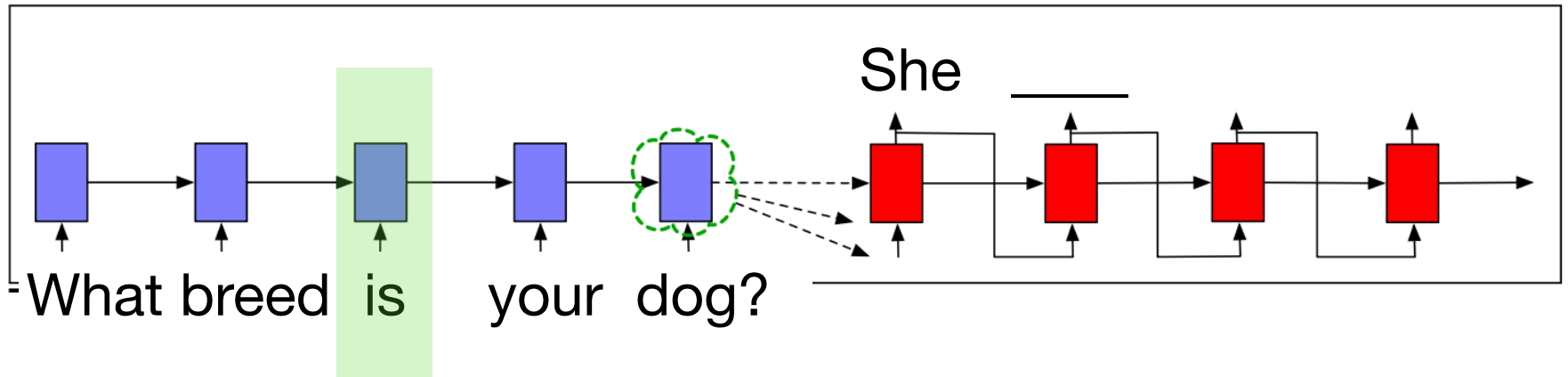
ICLR 2015

Which inputs matter?



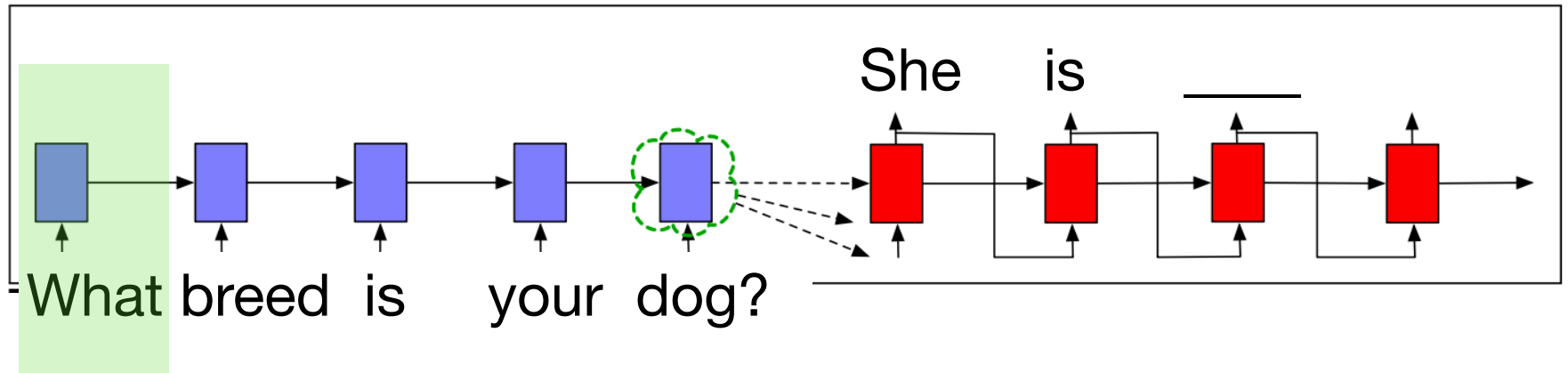
To complete this sentence, we probably want to pay **attention** to certain words in the input.

Which inputs matter?



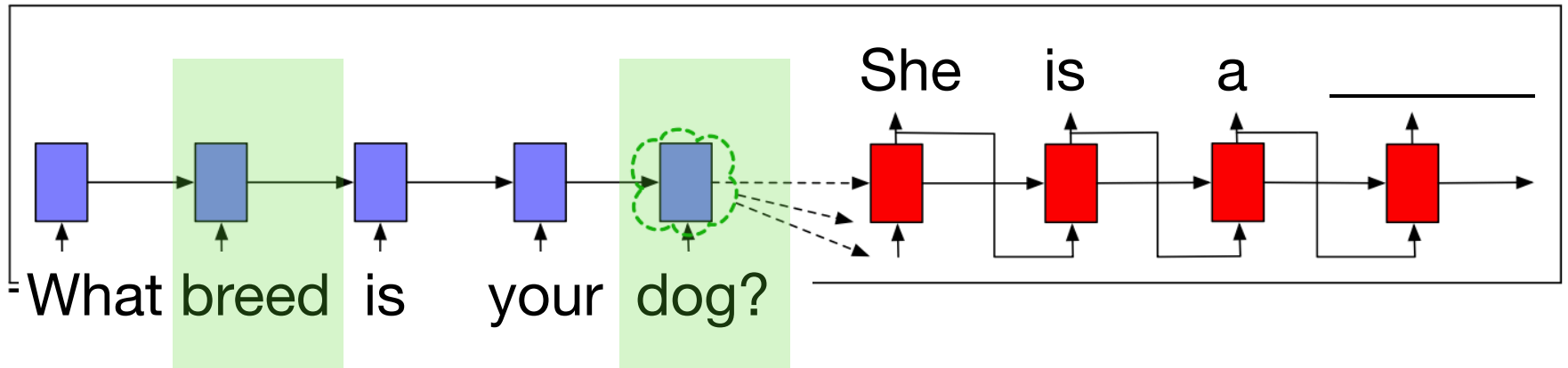
To complete this sentence, we probably want to pay **attention** to certain words in the input.

Which inputs matter?



To complete this sentence, we probably want to pay **attention** to certain words in the input.

Which inputs matter?

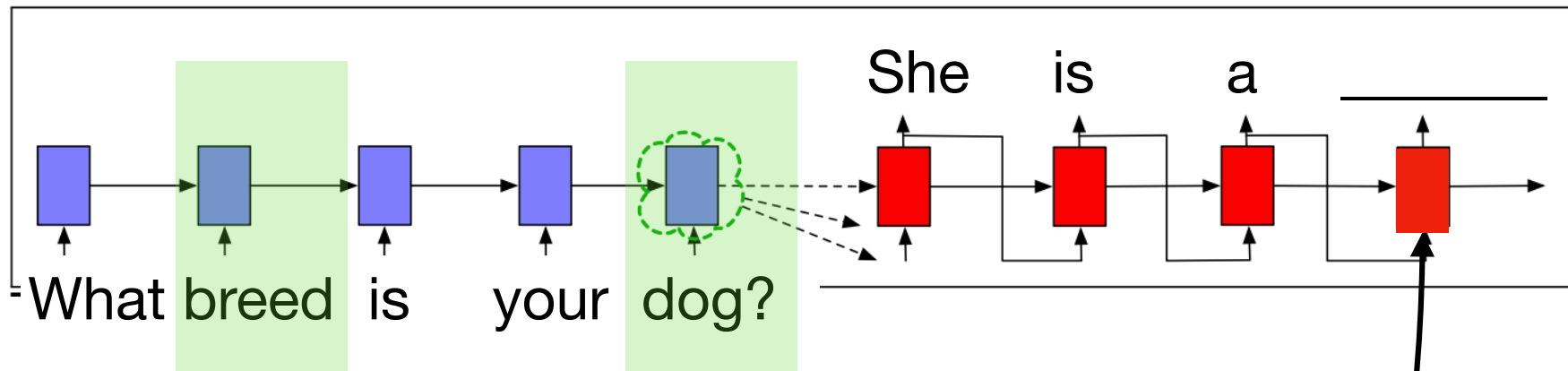


To complete this sentence, we probably want to pay **attention** to certain words in the input.

How can we mathematically define “attention”?

- (1) Given a vector representing “attention,”
how do I summarize the input?
- (2) How do I compute the attention vector?

Given an attention vector, how to summarize inputs?



Hidden
layer:

h_1^e h_2^e h_3^e h_4^e h_5^e

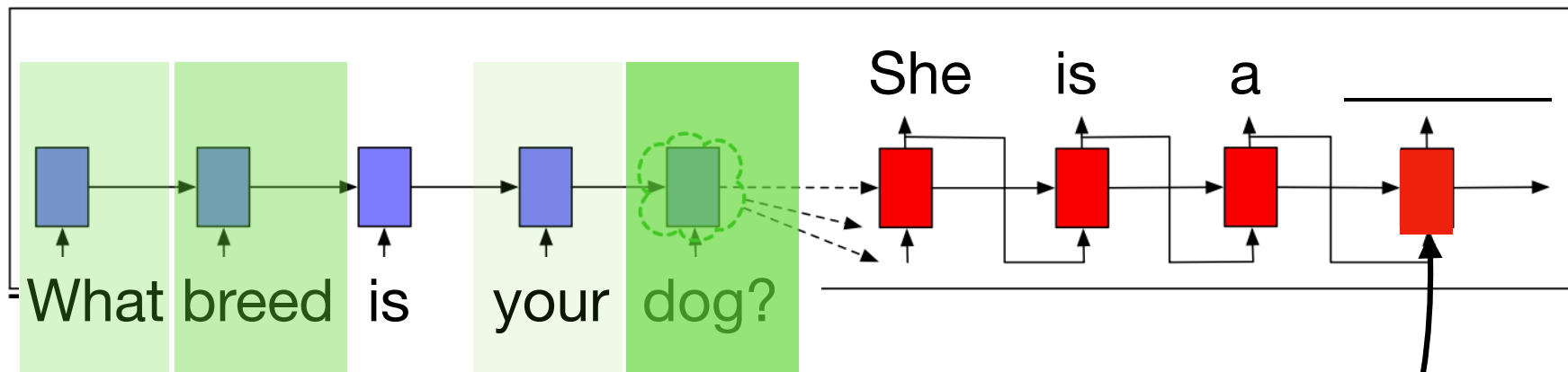
Attention
vector at
pos 4:

χ 0, 1, 0, 0, 1

$$0 + h_2^e + 0 + 0 + h_5^e = h_2^e + h_5^e$$

*Does not depend on
number of input tokens*

Given an attention vector, how to summarize inputs?



Hidden
layer:

h_1^e h_2^e h_3^e h_4^e h_5^e

Attention
vector at
pos 4:

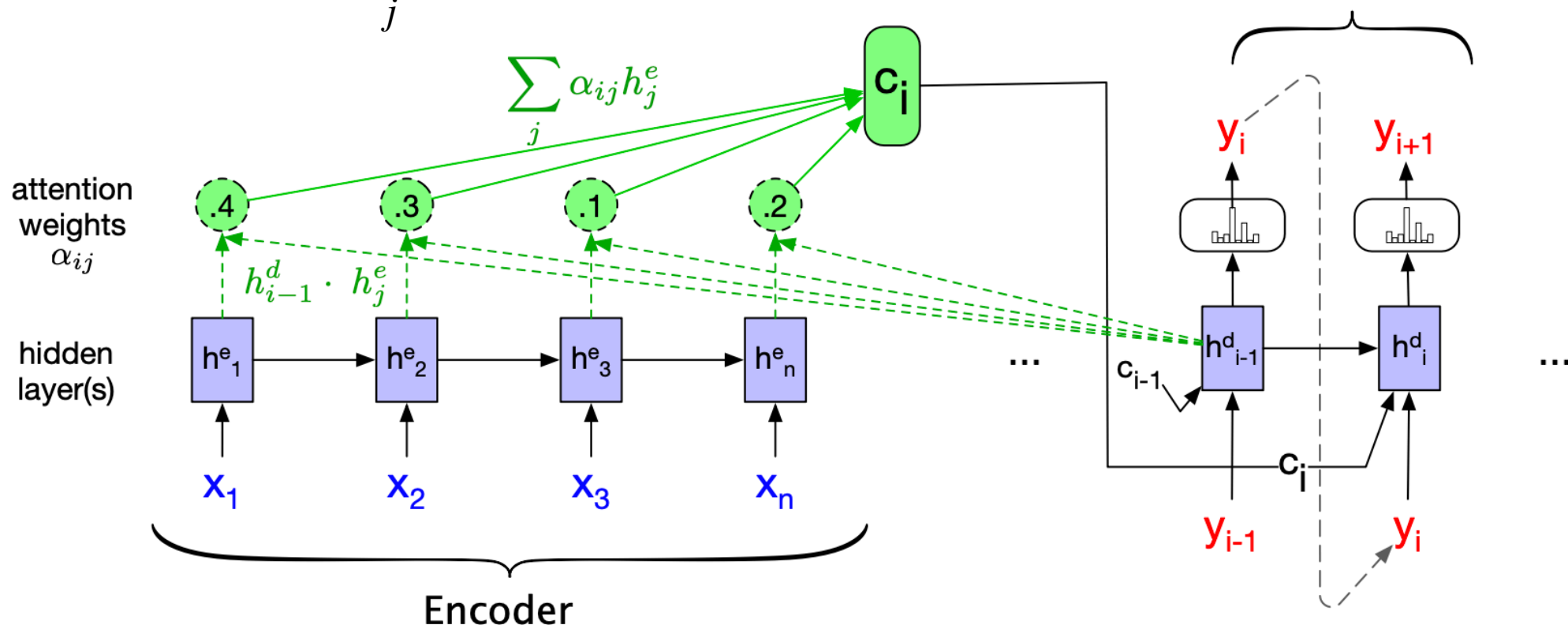
χ 0.2, 0.3, 0, 0.1, 0.4

$$0.2h_1^e + 0.3h_2^e + 0 + 0.1h_4^e + 0.4h_5^e$$

Given an attention vector, how to summarize inputs?

Denote the hidden layer for input token i by h_i^e .

Given attention vector α_t for position t , a summary of the relevant input words is $\sum_j \alpha_{tj} h_j^e$.



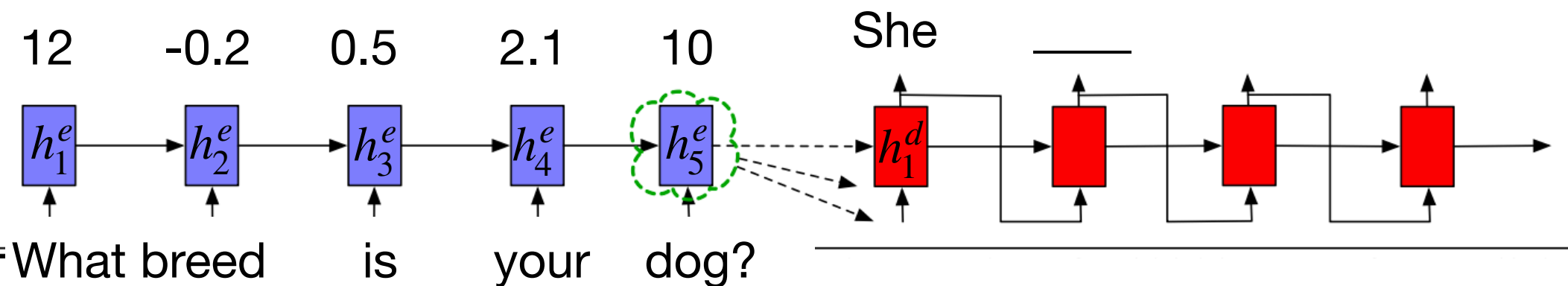
How can we mathematically define “attention”?

- ✓ (1) Given a vector representing “attention,”
how do I summarize the input?
- (2) How do I compute the attention vector?

How to compute attention?

- One way to compute attention is **dot-product similarity**:

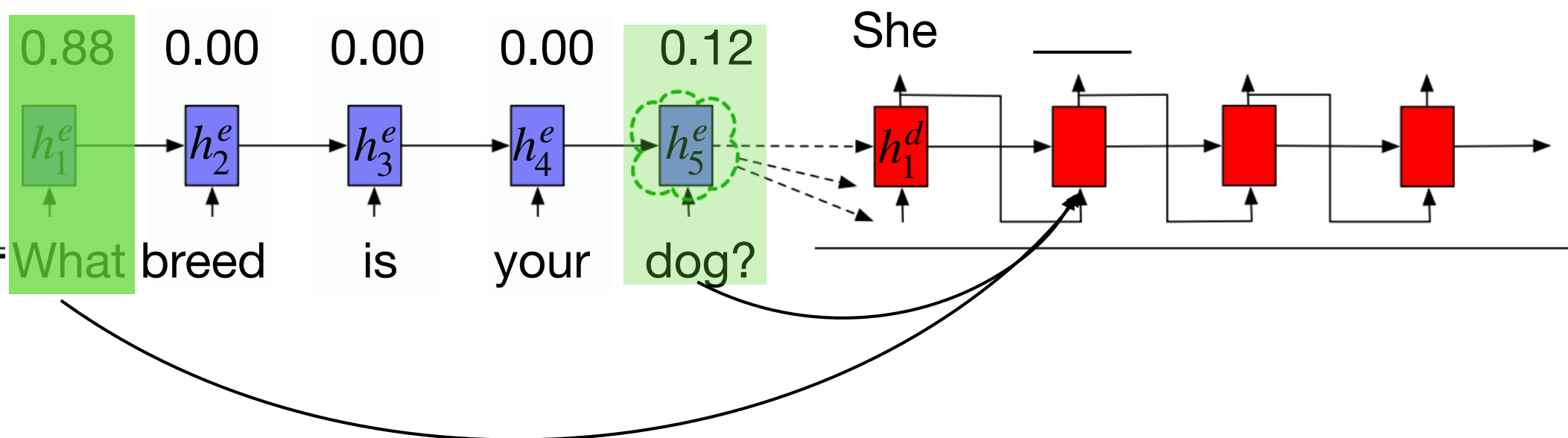
$$\alpha_{tj} = \text{softmax} \left(h_j^{e\top} h_{t-1}^d \right)$$



How to compute attention?

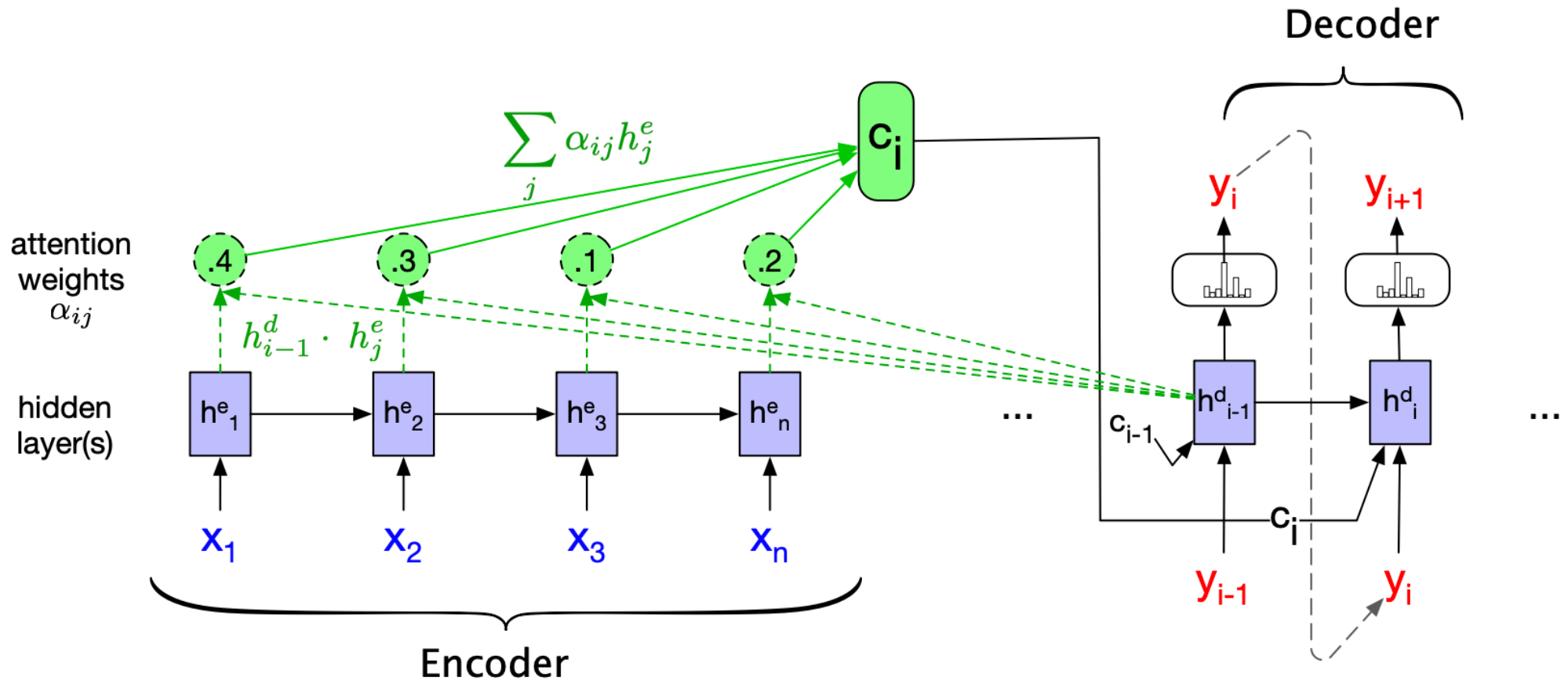
- One way to compute attention is **dot-product similarity**:

$$\alpha_{tj} = \text{softmax} \left(h_j^{e\top} h_{t-1}^d \right)$$



How can we mathematically define “attention”?

- ✓ (1) Given a vector representing “attention,” how do I summarize the input?
- ✓ (2) How do I compute the attention vector?



Attention

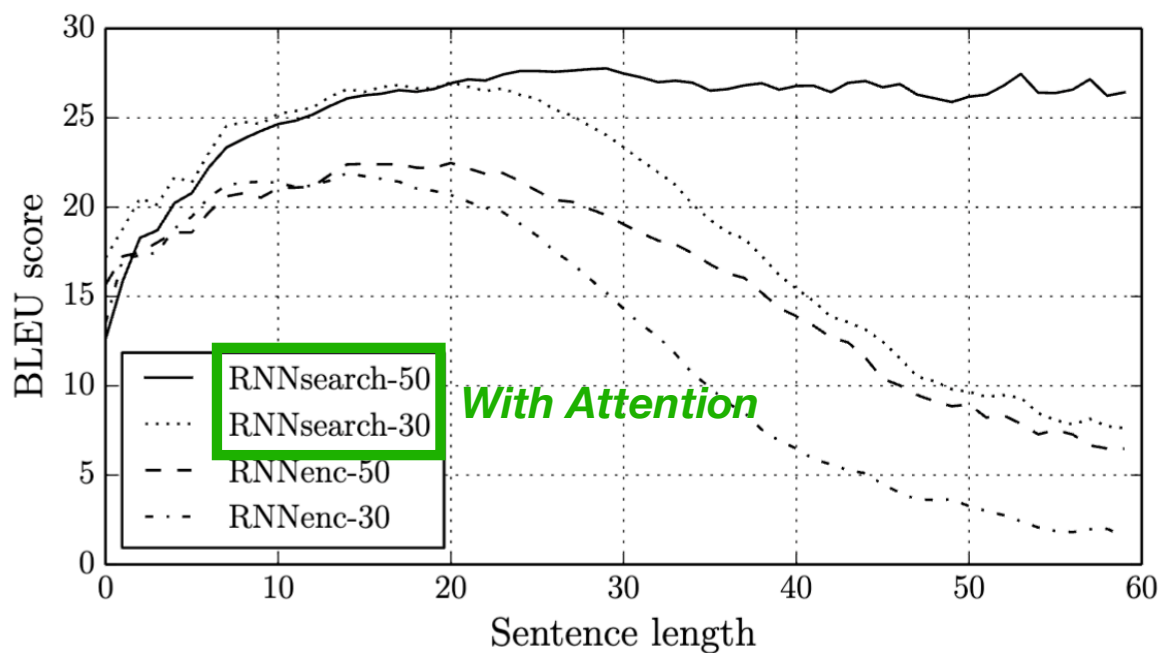
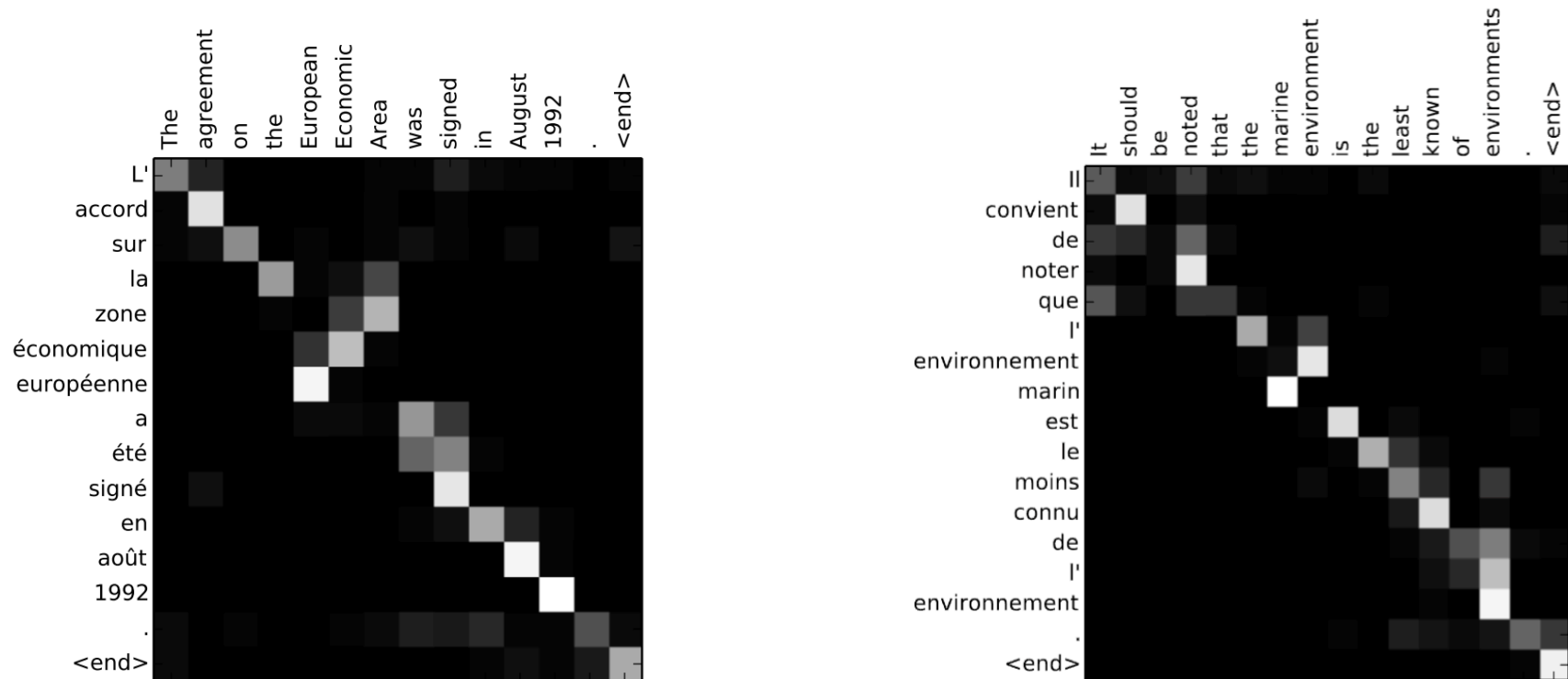


Figure 2: The BLEU scores of the generated translations on the test set with respect to the lengths of the sentences. The results are on the full test set which includes sentences having unknown words to the models.

Attention: visualization



Scalable and accurate deep learning with electronic health records

Alvin Rajkomar^{1,2}, Eyal Oren¹, Kai Chen¹, Andrew M. Dai¹, Nissan Hajaj¹, Michaela Hardt¹, Peter J. Liu¹, Xiaobing Liu¹, Jake Marcus¹, Mimi Sun¹, Patrik Sundberg¹, Hector Yee¹, Kun Zhang¹, Yi Zhang¹, Gerardo Flores¹, Gavin E. Duggan¹, Jamie Irvine¹, Quoc Le¹, Kurt Litsch¹, Alexander Mossin¹, Justin Tansuwan¹, De Wang¹, James Wexler¹, Jimbo Wilson¹, Dana Ludwig², Samuel L. Volchenboum³, Katherine Chou¹, Michael Pearson¹, Srinivasan Madabushi¹, Nigam H. Shah⁴, Atul J. Butte², Michael D. Howell¹, Claire Cui¹, Greg S. Corrado¹ and Jeffrey Dean¹

npj | Digital Medicine

*Interpreting
attention?*

Patient Timeline

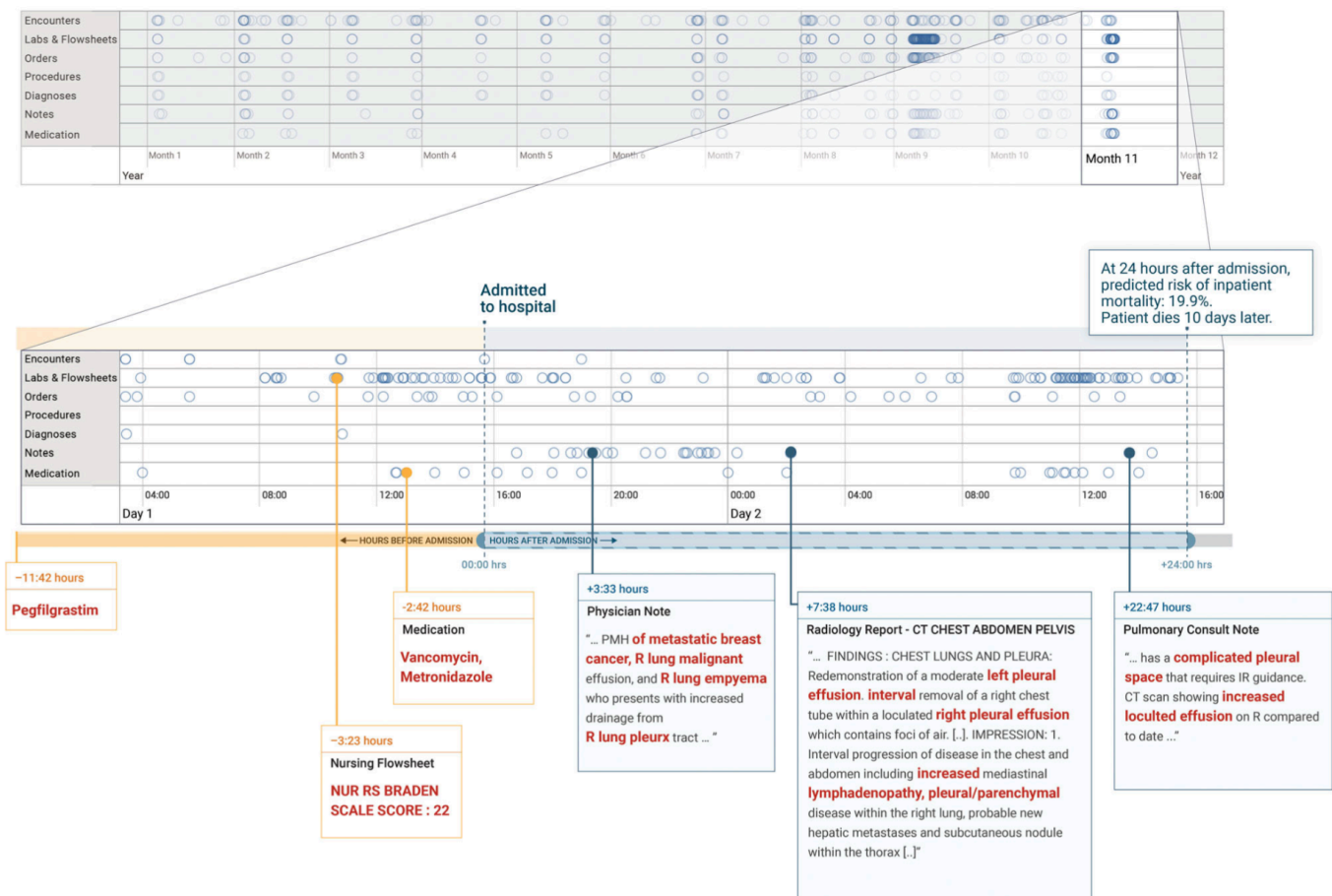


Fig. 3 The patient record shows a woman with metastatic breast cancer with malignant pleural effusions and empyema. The patient timeline at the top of the figure contains circles for every time-step for which at least a single token exists for the patient, and the horizontal lines show the data type. There is a close-up view of the most recent data points immediately preceding a prediction made 24 h after admission. We trained models for each data type and highlighted in red the tokens which the models attended to—the non-highlighted text was not attended to but is shown for context. The models pick up features in the medications, nursing flowsheets, and clinical notes relevant to the prediction

But is attention an explanation?

after 15 minutes watching the movie i was asking myself what to do leave the theater sleep or try to keep watching the movie to see if there was anything worth i finally watched the movie what a waste of time maybe i am not a 5 years old kid anymore

original α

$$f(x|\alpha, \theta) = 0.01$$

after 15 minutes watching the movie i was asking myself what to do leave the theater sleep or try to keep watching the movie to see if there was anything worth i finally watched the movie what a waste of time maybe i am not a 5 years old kid anymore

adversarial $\tilde{\alpha}$

$$f(x|\tilde{\alpha}, \theta) = 0.01$$

Figure 1: Heatmap of attention weights induced over a negative movie review. We show observed model attention (left) and an adversarially constructed set of attention weights (right). Despite being quite dissimilar, these both yield effectively the same prediction (0.01).

Lecture Outline

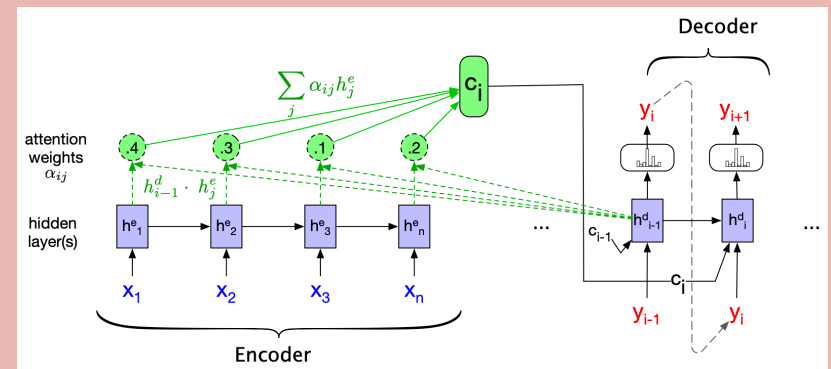
- Encoder-Decoder Network
- Attention mechanism
- **Transformers**
- Generative Pretrained Transformers (GPT)

Inductive biases in NN

- Until very recently, the assumption was that we need inductive biases in the NN to help it generalize.
- **Convolutional neural networks:** Encodes translational invariance
- **Recurrent neural networks:** Also encodes translational invariance, but with memory

Problems with these models:

- *Computation is slow*



- *Learning dependencies between distant positions is difficult*

What if we threw it all out the window?

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

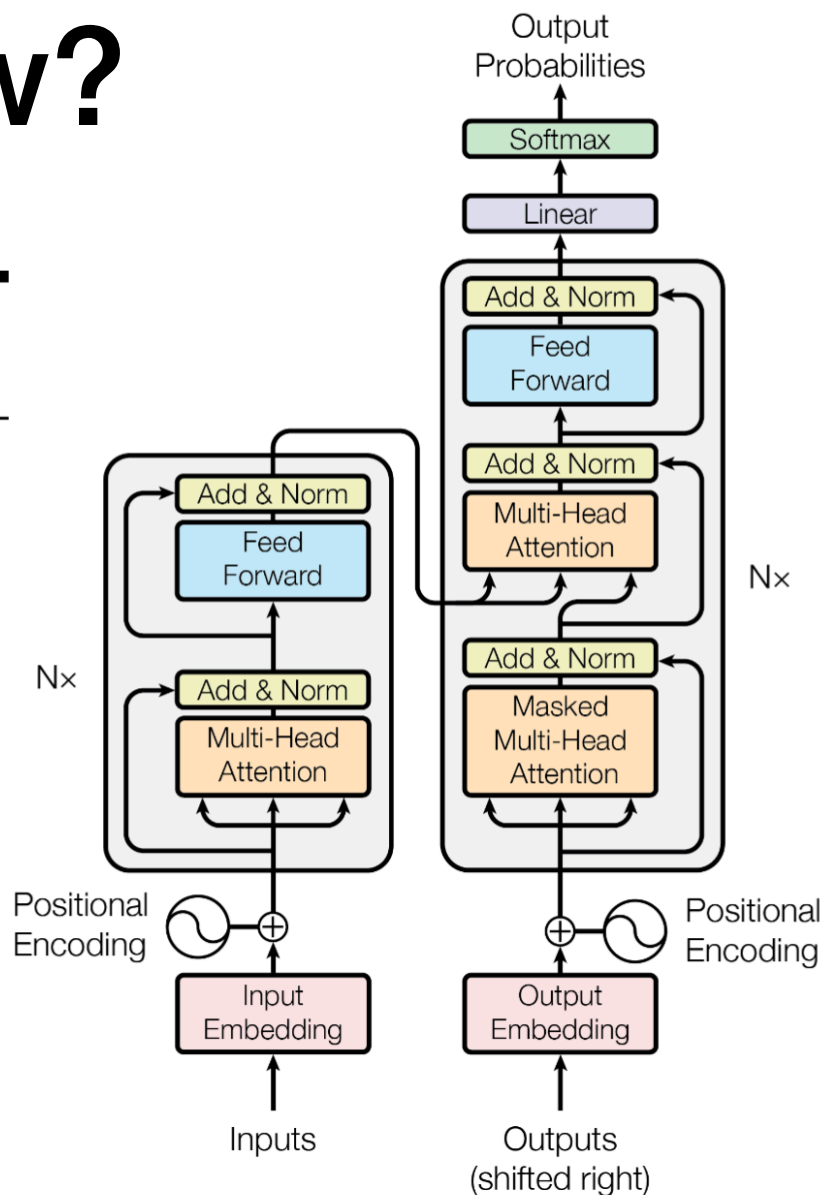
Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Essentially zero inductive biases



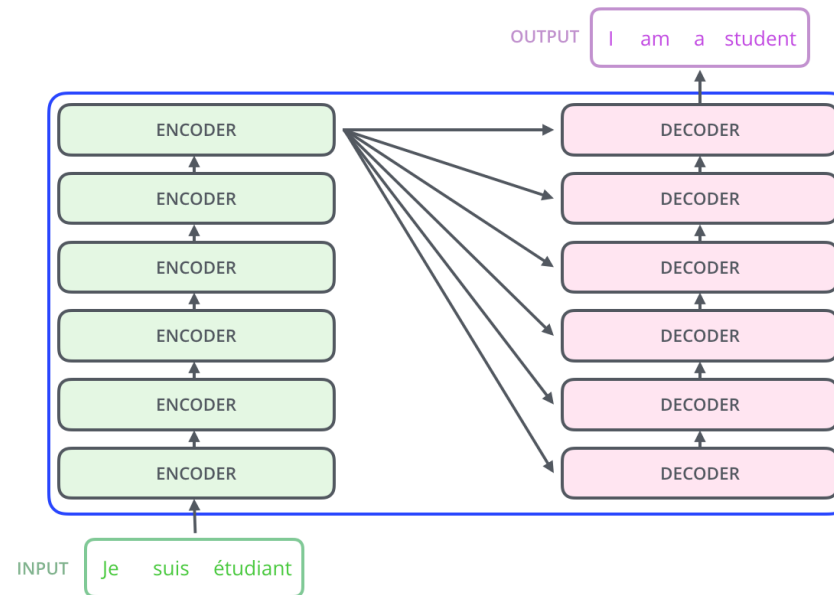
Shoot, it works well.

We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence pairs. Sentences were encoded using byte-pair encoding [3], which has a shared source-target vocabulary of about 37000 tokens. For English-French, we used the significantly larger WMT 2014 English-French dataset consisting of 36M sentences and split tokens into a 32000 word-piece vocabulary [31]. Sentence pairs were batched together by approximate sequence length. Each training batch contained a set of sentence pairs containing approximately 25000 source tokens and 25000 target tokens.

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [15]	23.75			
Deep-Att + PosUnk [32]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [31]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [8]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [26]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [32]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [31]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [8]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.0	$2.3 \cdot 10^{19}$	

Understanding transformers



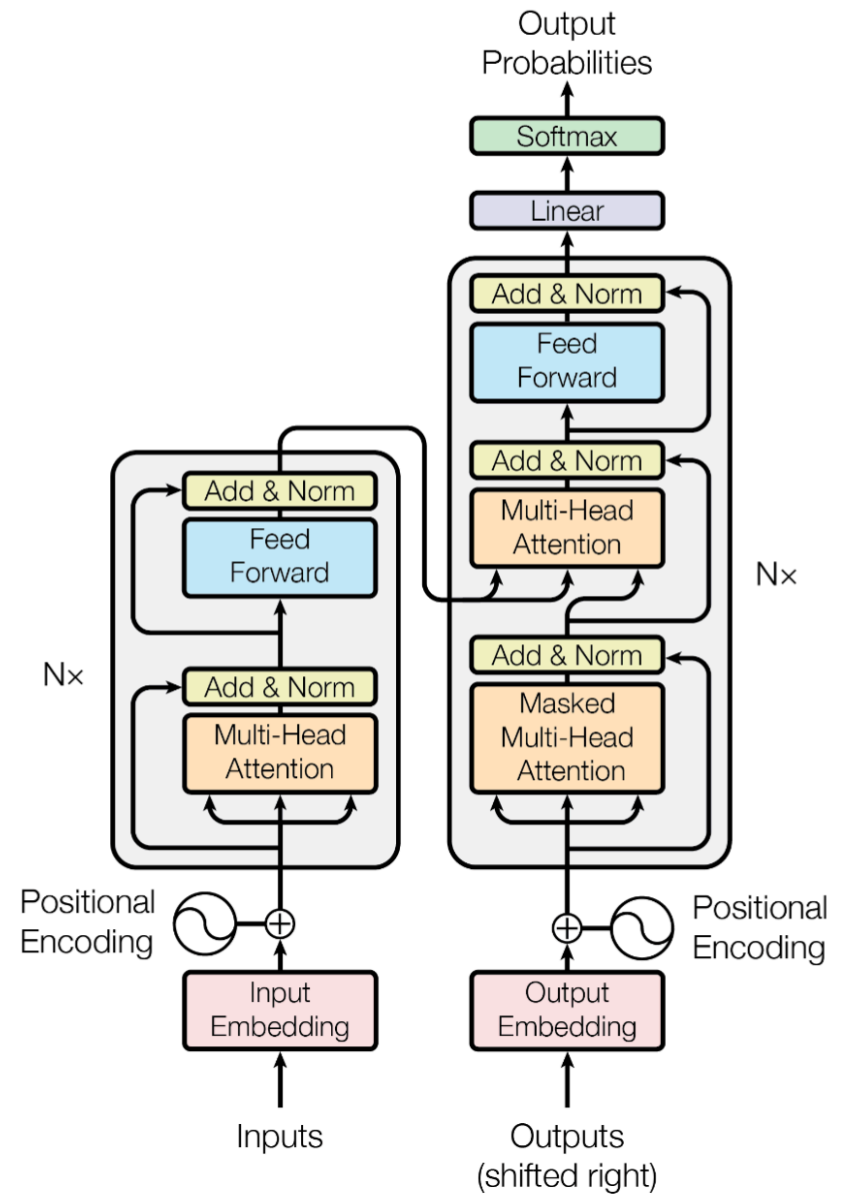
<https://jalammar.github.io/illustrated-transformer/>

Parts of a transformer:

- Tokenization
- Input embeddings✓
- Position encodings
- Query, key, value
- Attention✓
- Self attention
- Multi-head attention
- Feed-forward✓
- Encoders✓
- Decoders✓
- Residual connections✓
- Layer norms

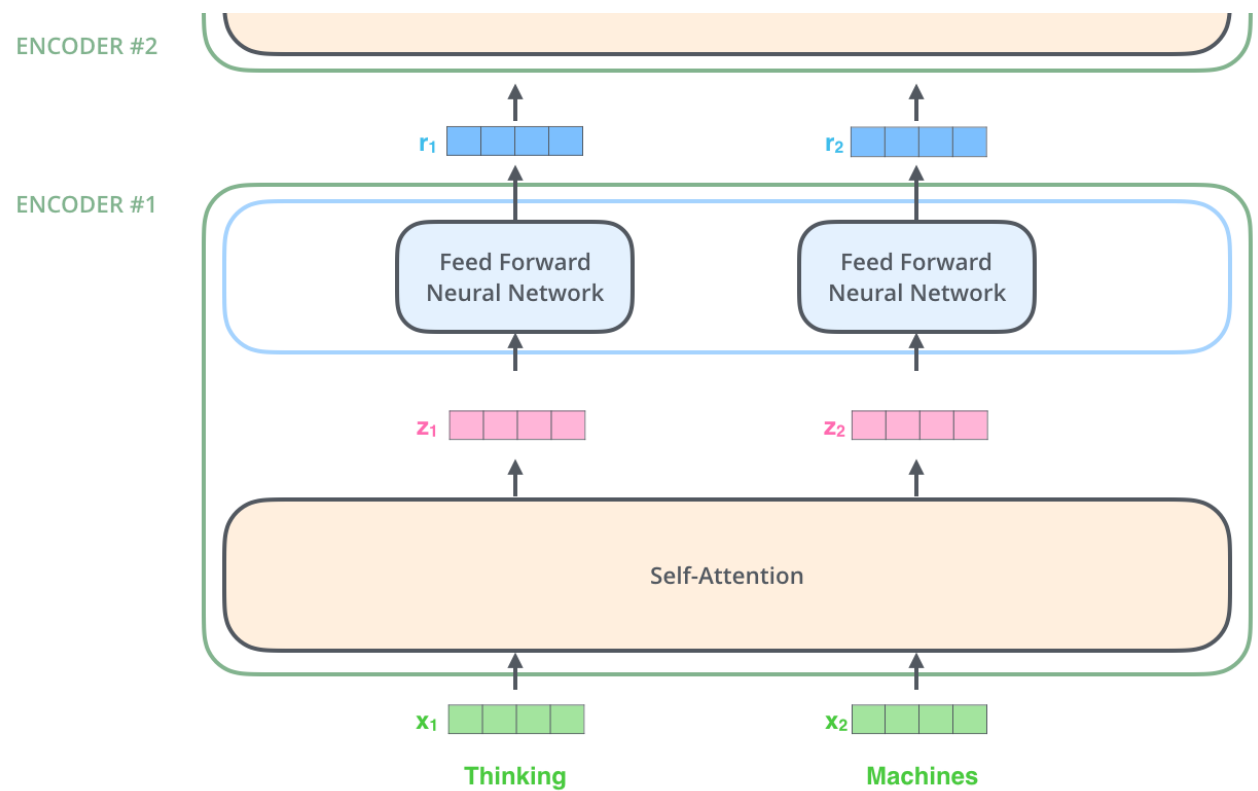
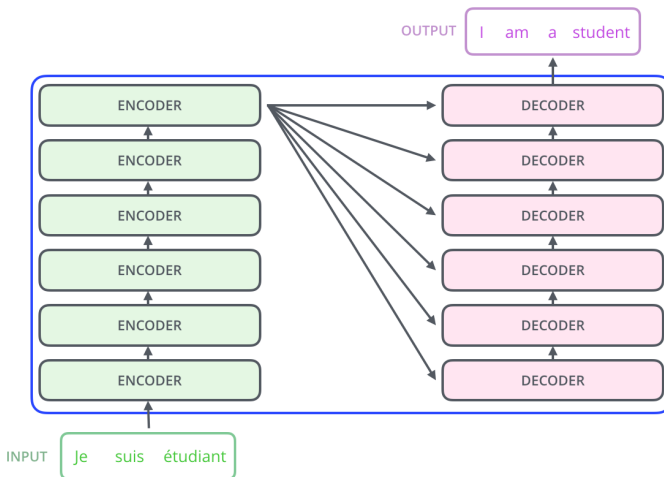
Quiz

- Where in the transformer do residual connections appear?

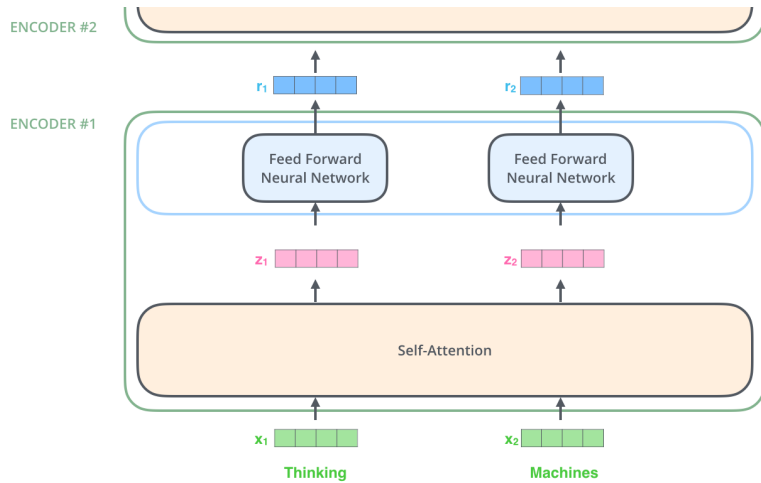


Understanding transformers:

The encoder part



Understanding transformers: Self-attention



(1) Make Queries, Keys, and Values

$$X \times W^Q = Q$$

A diagram showing the calculation of the Query matrix Q . It consists of a green 2x4 matrix X multiplied by a purple 4x3 matrix W^Q , resulting in a purple 2x3 matrix Q .

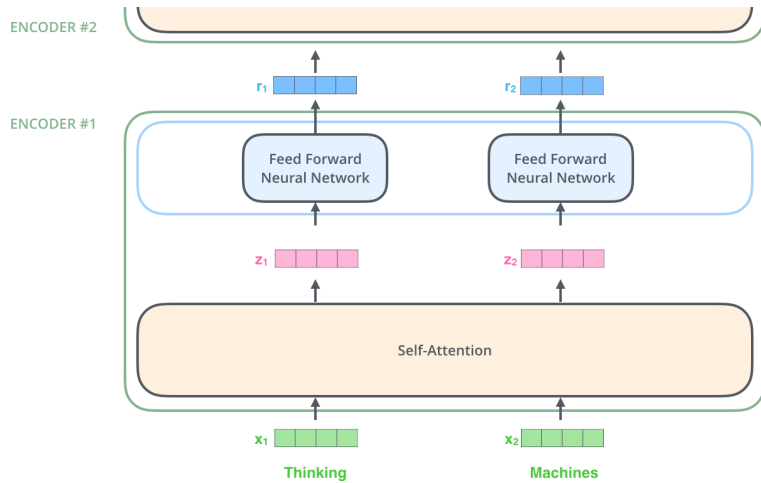
$$X \times W^K = K$$

A diagram showing the calculation of the Key matrix K . It consists of a green 2x4 matrix X multiplied by an orange 4x3 matrix W^K , resulting in an orange 2x3 matrix K .

$$X \times W^V = V$$

A diagram showing the calculation of the Value matrix V . It consists of a green 2x4 matrix X multiplied by a blue 4x3 matrix W^V , resulting in a blue 2x3 matrix V .

Understanding transformers: Self-attention



(1) Make Queries, Keys, and Values

Input

Thinking

Machines

Embedding

x_1

x_2

Queries

q_1

q_2

W^Q

Keys

k_1

k_2

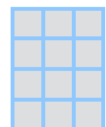
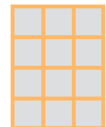
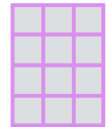
W^K

Values

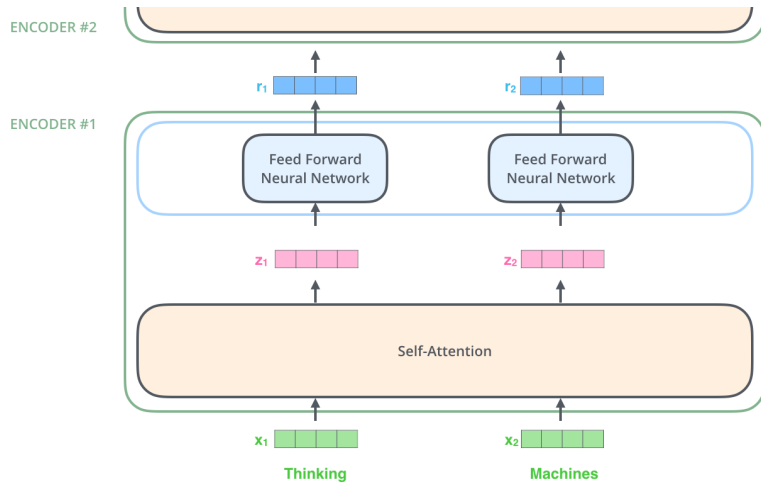
v_1

v_2

W^V



Understanding transformers: Self-attention



(2) Compute attention

$$\text{softmax} \left(\frac{\begin{matrix} \text{Q} \\ \begin{matrix} \text{3x3 grid} \end{matrix} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{matrix} \text{3x3 grid} \end{matrix} \end{matrix}}{\sqrt{d_k}} \right)$$

Input

Embedding

Queries

Keys

Values

Score

Divide by 8 ($\sqrt{d_k}$)

Softmax

Thinking

x_1

q_1

k_1

v_1

$q_1 \cdot k_1 = 112$

14

0.88

Machines

x_2

q_2

k_2

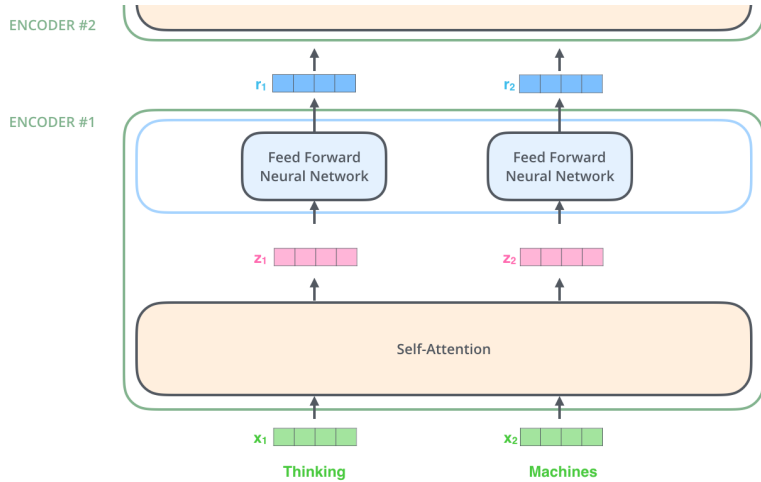
v_2

$q_1 \cdot k_2 = 96$

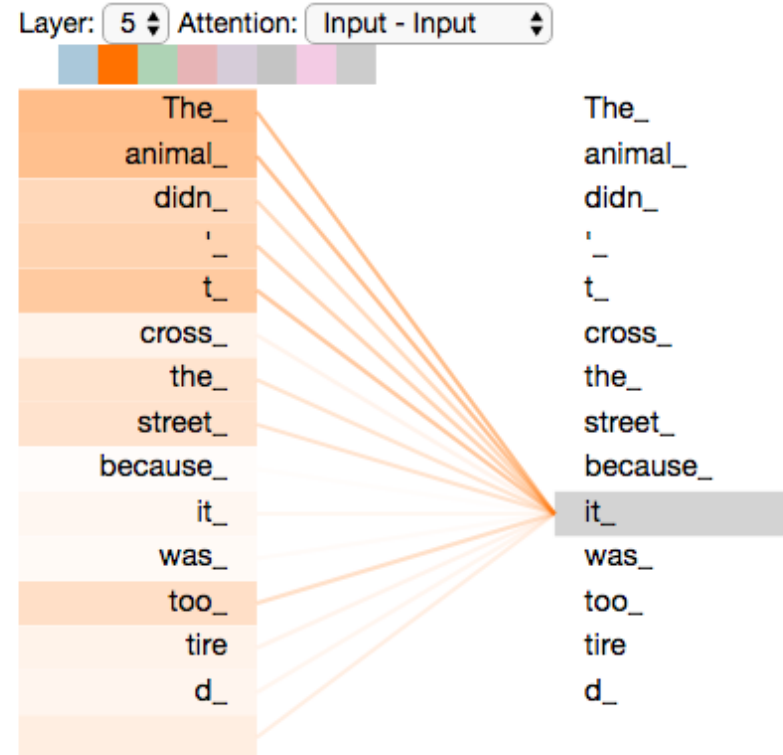
12

0.12

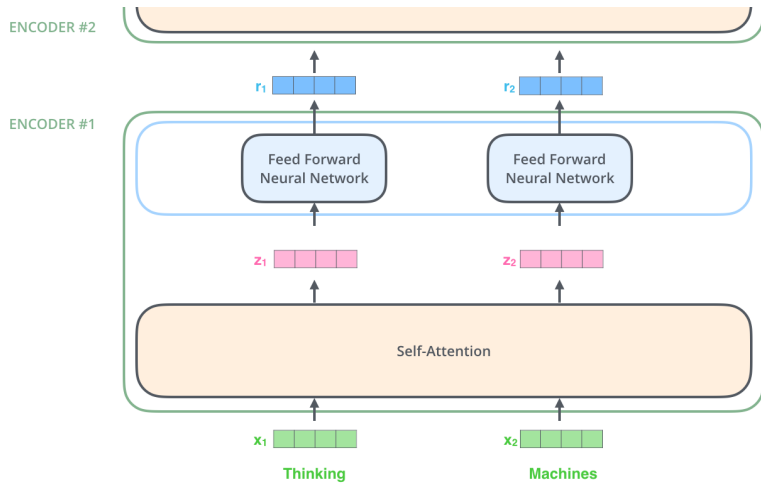
Understanding transformers: Self-attention



(2) Compute attention



Understanding transformers: Self-attention

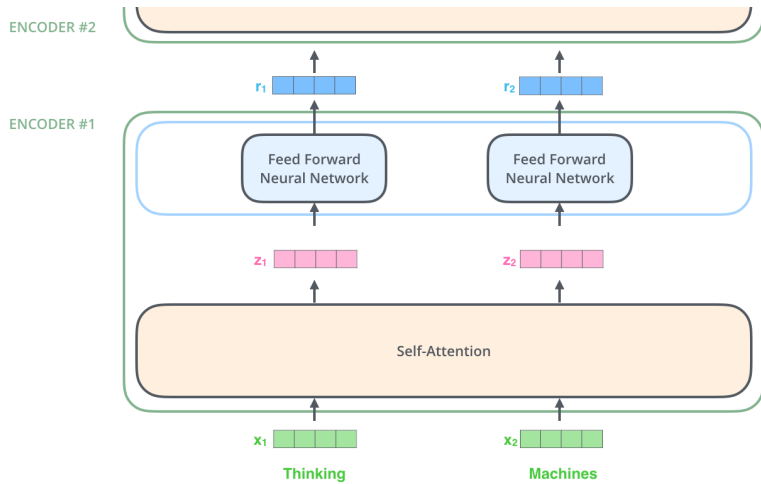


(3) Selective extraction of information based on attention

$$\text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \begin{matrix} \text{3x3 grid} \end{matrix} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{matrix} \text{3x3 grid} \end{matrix} \end{matrix}}{\sqrt{d_k}}\right) \begin{matrix} \text{V} \\ \begin{matrix} \text{3x3 grid} \end{matrix} \end{matrix} = \begin{matrix} \text{Z} \\ \begin{matrix} \text{3x3 grid} \end{matrix} \end{matrix}$$

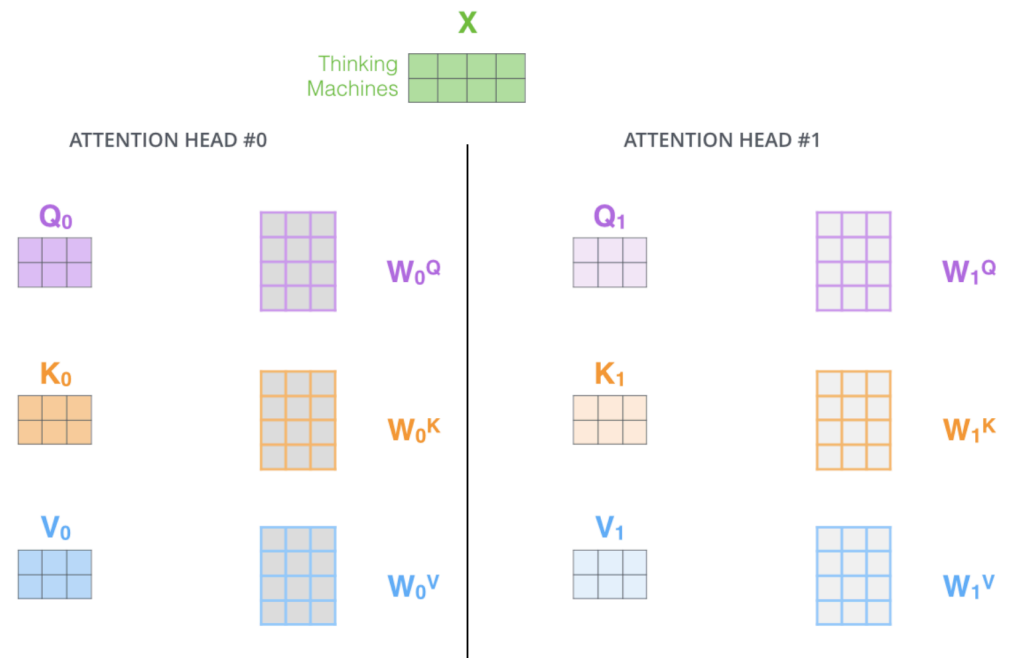
Understanding transformers:

Multi-headed attention



(4) Need to summarize other information that require different attention patterns?

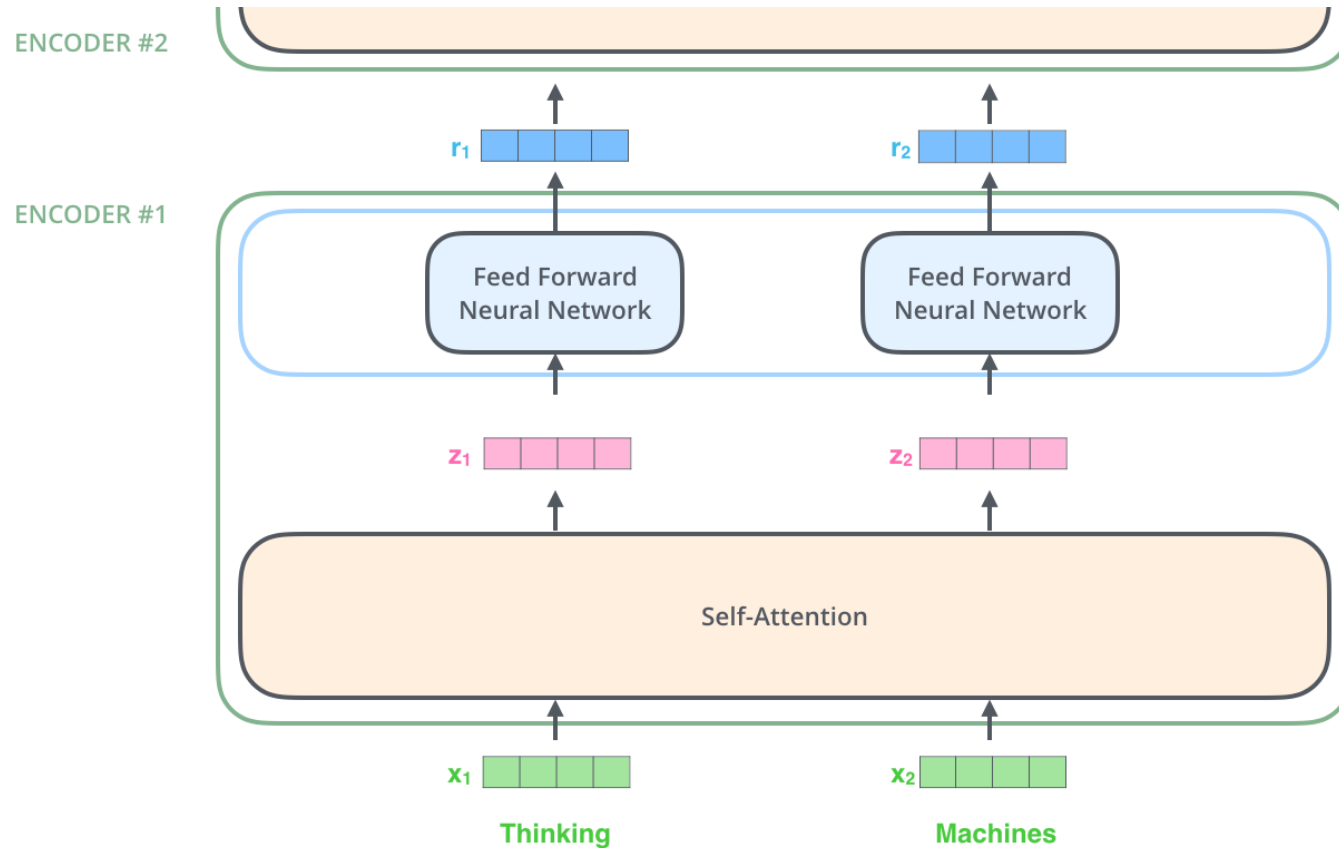
Use multiple attention heads!



Understanding transformers:

Multi-headed attention

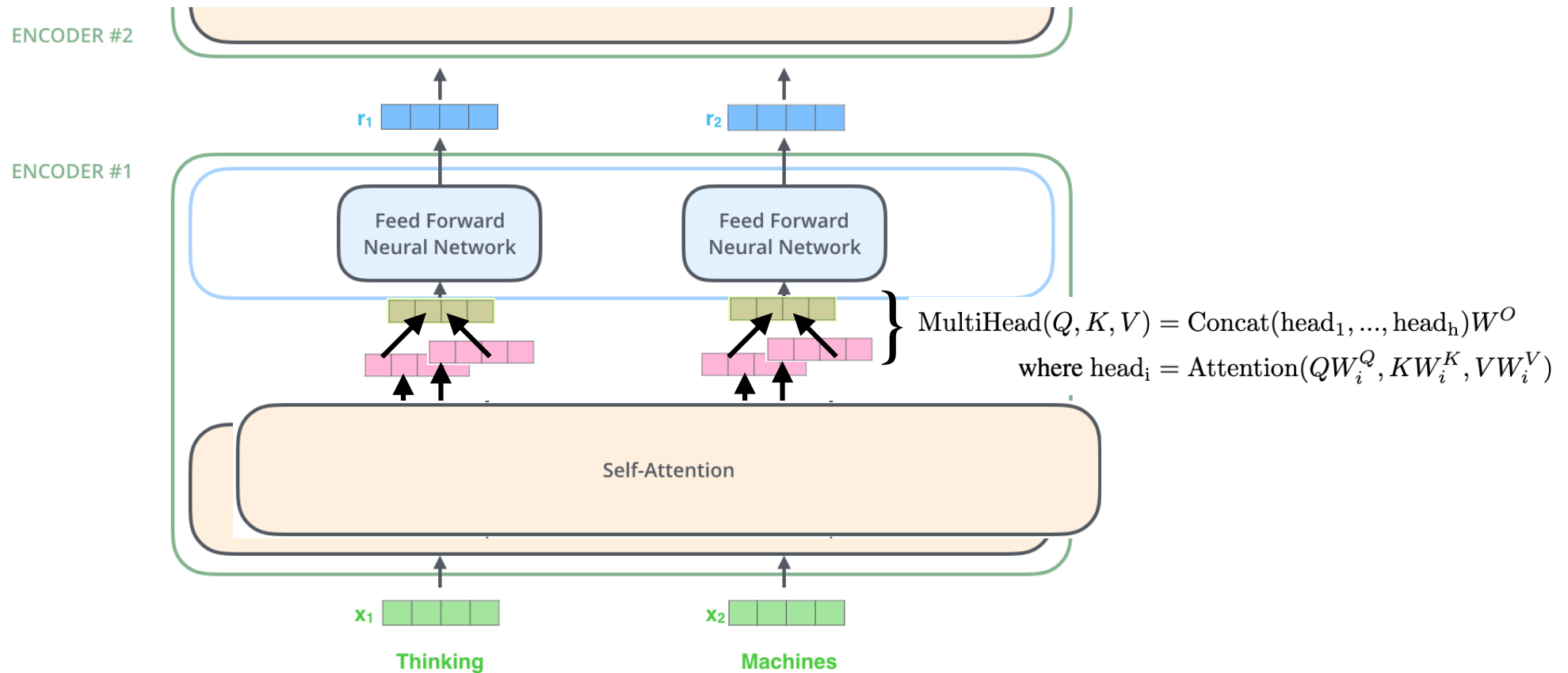
(4) Combining information from multi-headed attention



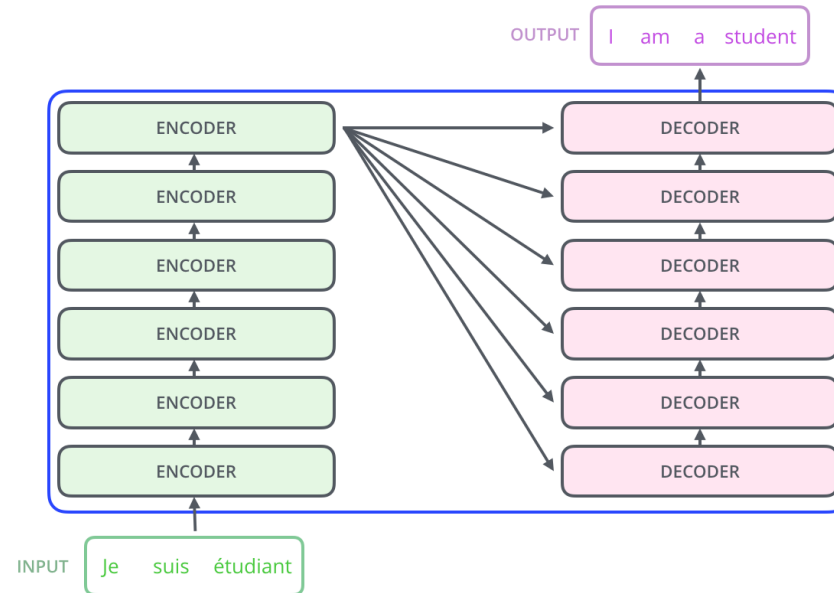
Understanding transformers:

Multi-headed attention

(4) Combining information from multi-headed attention



Understanding transformers



<https://jalammar.github.io/illustrated-transformer/>

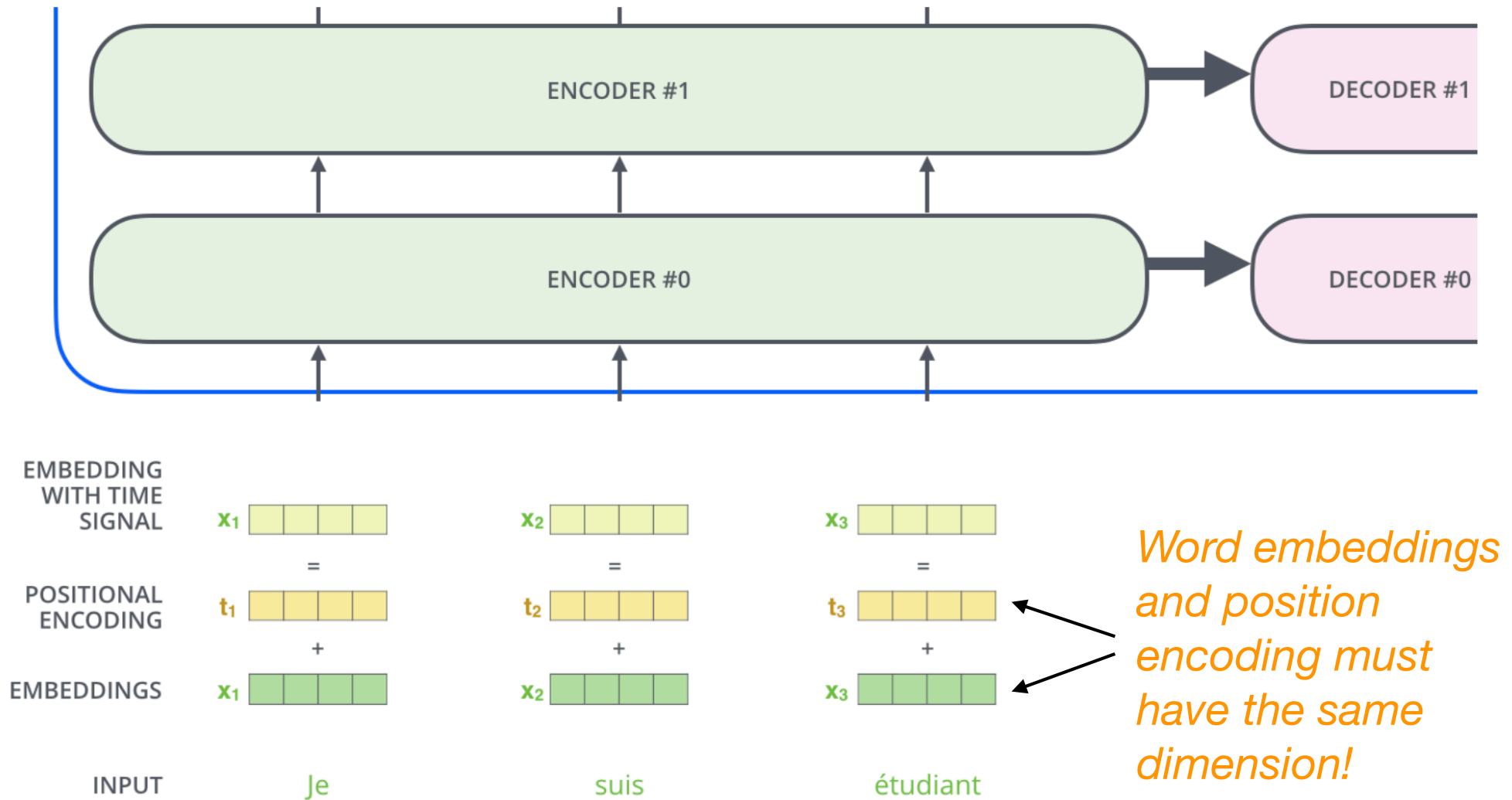
Parts of a transformer:

- Tokenization
- Input embeddings ✓
- Position encodings
- Query, key, value ✓
- Attention ✓
- Self attention ✓
- Multi-head attention ✓
- Feed-forward ✓
- Encoders ✓
- Decoders ✓
- Residual connections ✓
- Layer norms

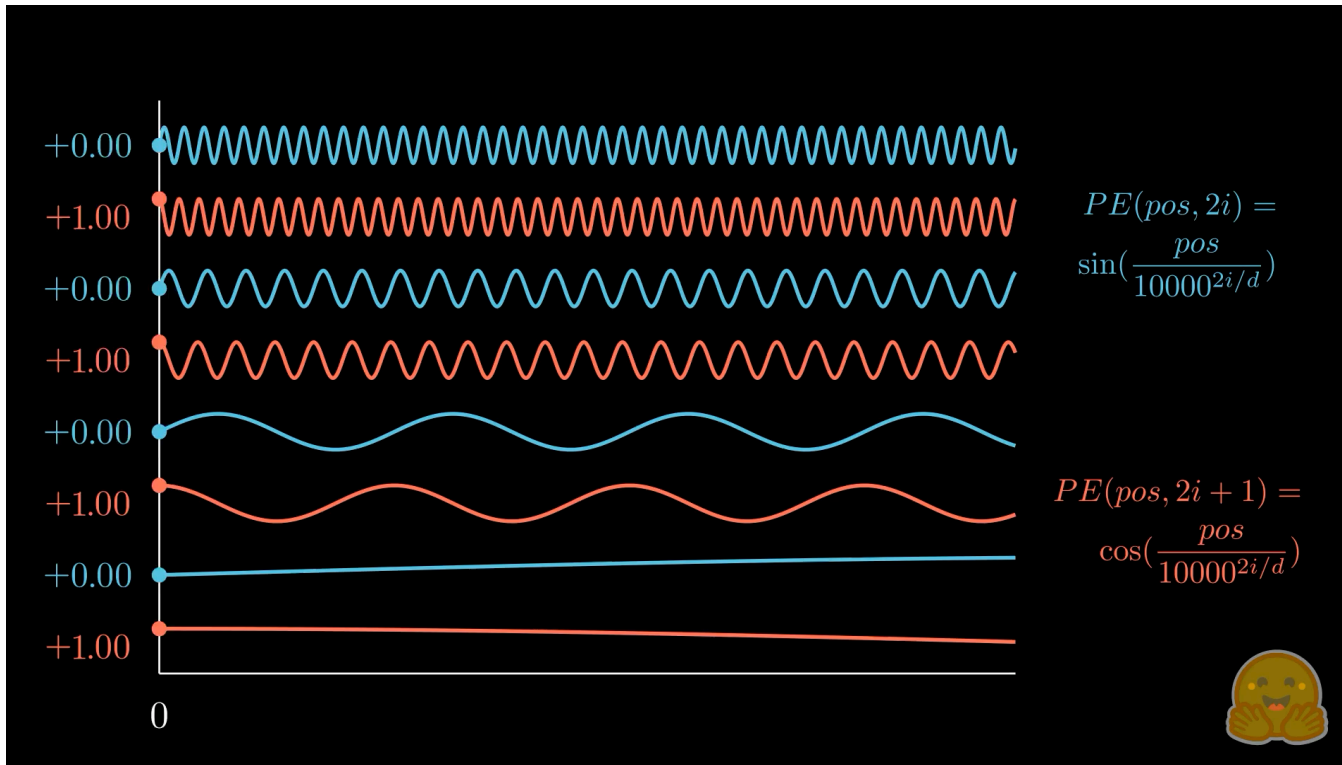
Position encoding

- Because the transformer does not use recurrence or convolution, it does not know anything about the position of a word in a sentence unless we provide this information explicitly!
- Position of a word (ordering of words) in a sentence is important, so we need to figure out how to pass in this information!
- *Any ideas for how we might encode position information?*

Position encoding



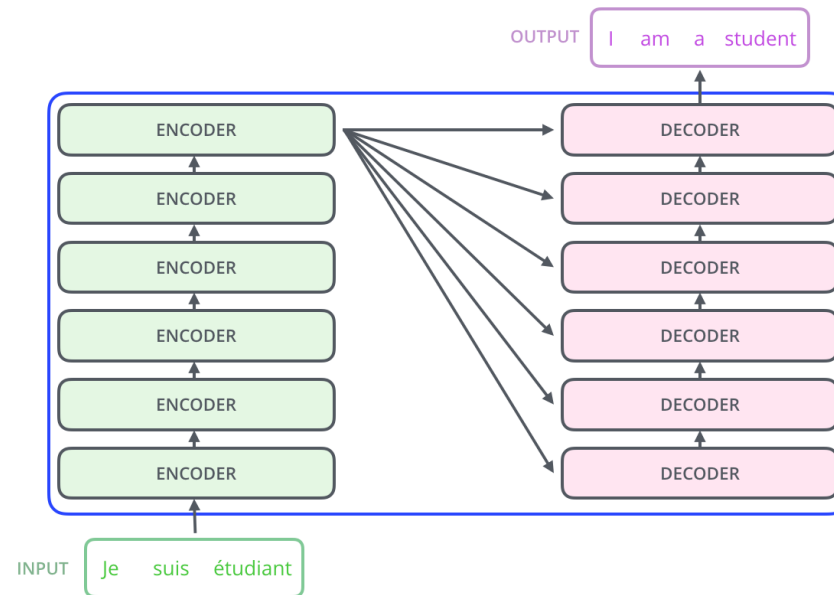
Position encoding



$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

where pos =position
and i equals
dimension of the
embedding

Understanding transformers

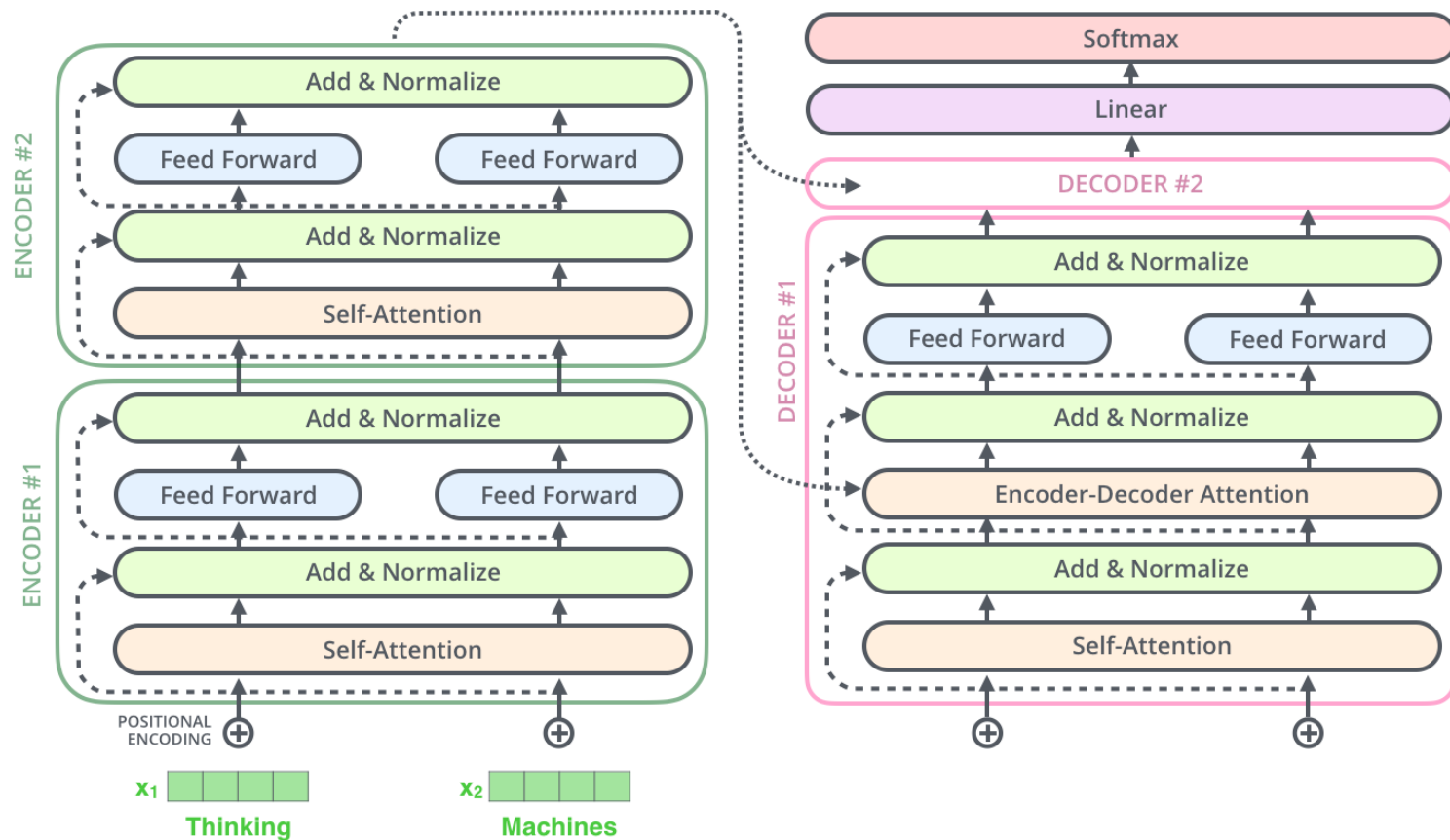


<https://jalammar.github.io/illustrated-transformer/>

Parts of a transformer:

- Tokenization
- Input embeddings ✓
- Position encodings ✓
- Query, key, value ✓
- Attention ✓
- Self attention ✓
- Multi-head attention ✓
- Feed-forward ✓
- Encoders ✓
- Decoders ✓
- Residual connections ✓
- Layer norms

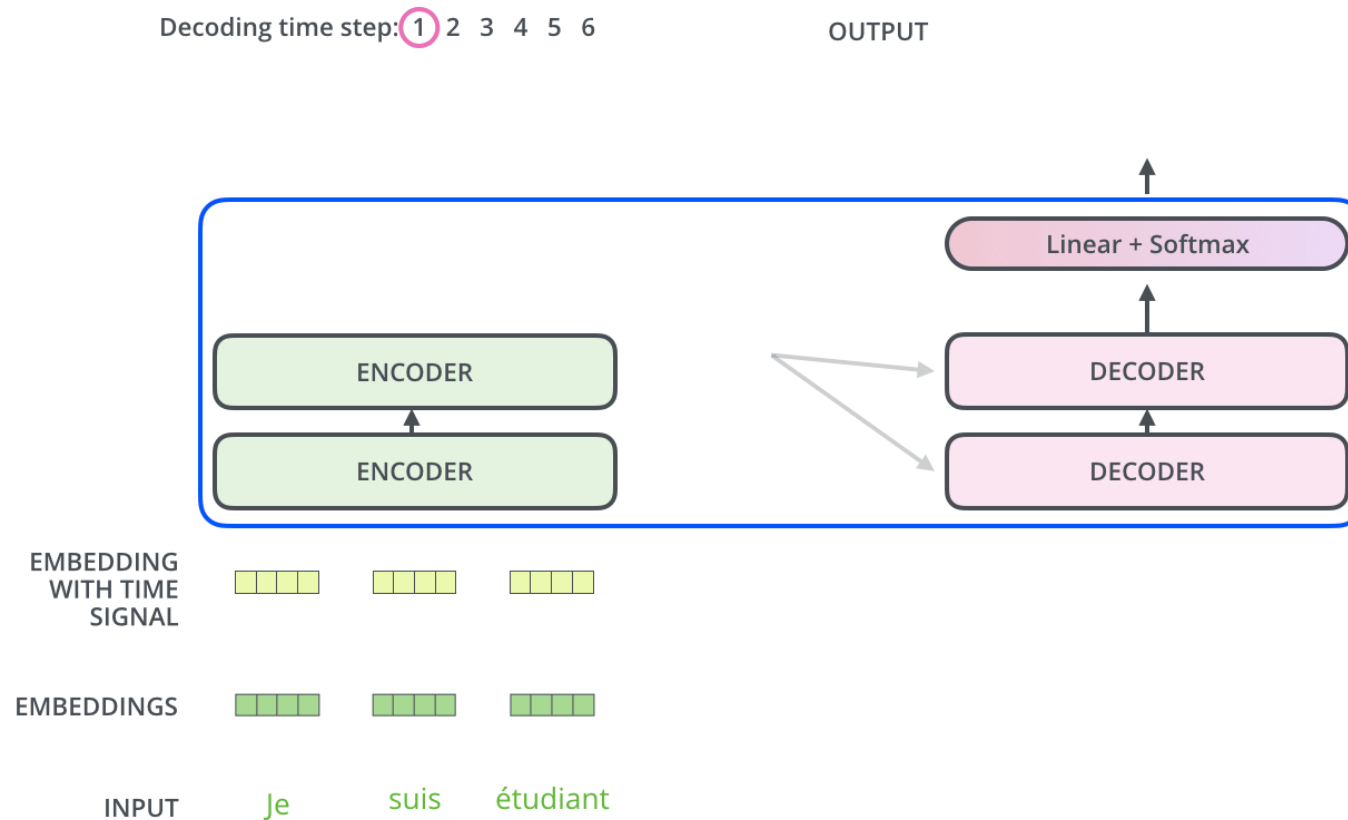
Understanding transformers: What it looks like with 2 encoders



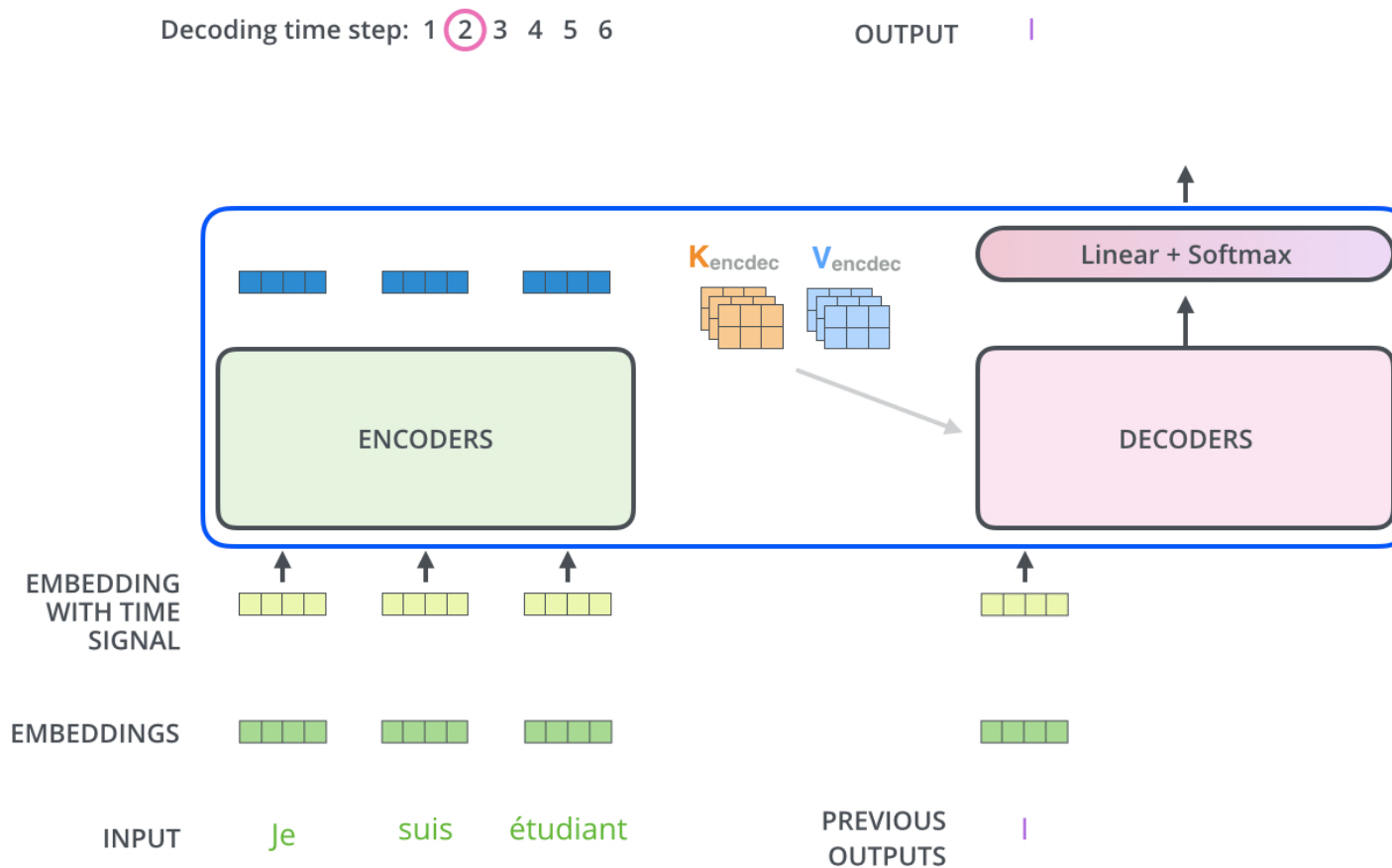
*That's all to encoders!
Moving on to decoders!*

Understanding transformers:

Decoding



Understanding transformers: Decoding



Transformers

- **Summary:**
- Transformers remove many of the inductive biases in NN and use attention modules instead. With sufficient training data, it can learn these inductive biases instead.
- Transformers have changed the field of DL drastically, outperforming many models that used to be State Of The Art (SOTA).

Food for thought

- What do you think a Vision Transformer does? How would you use extend the Transformer architecture to deal with images?

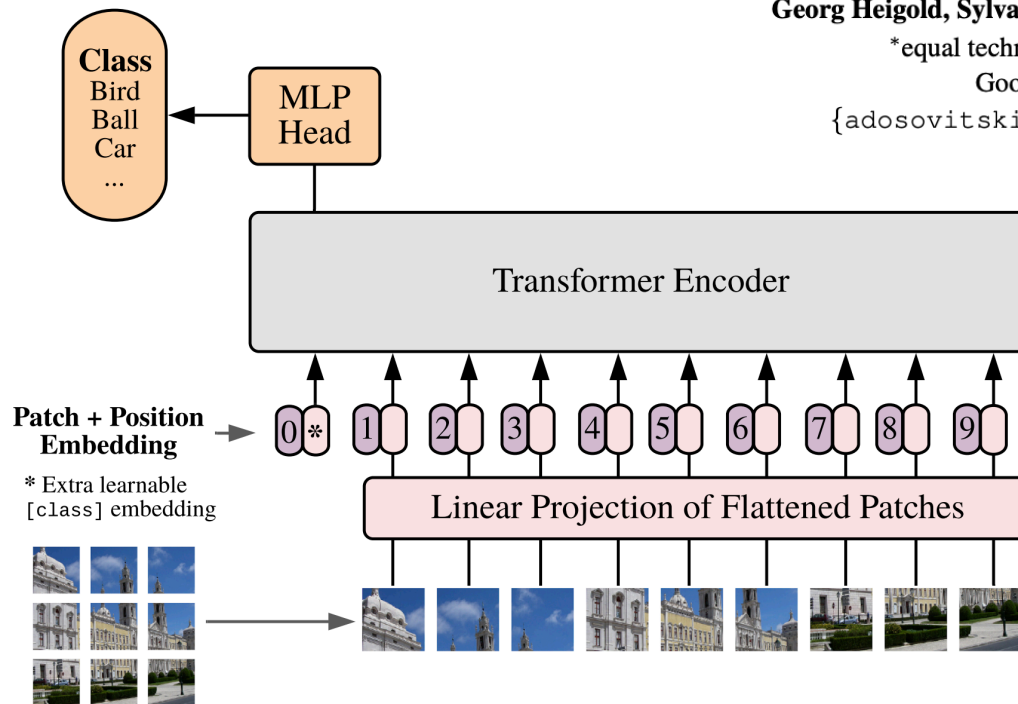
AN IMAGE IS WORTH 16X16 WORDS:
TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy^{*,†}, Lucas Beyer^{*}, Alexander Kolesnikov^{*}, Dirk Weissenborn^{*},
Xiaohua Zhai^{*}, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby^{*,†}

^{*}equal technical contribution, [†]equal advising

Google Research, Brain Team

{adosovitskiy, neilhoulby}@google.com



Lecture Outline

- Encoder-Decoder Network
- Attention mechanism
- Transformers
- **Generative Pretrained Transformers (GPT)**

From transformers to GPT

Improving Language Understanding by Generative Pre-Training

Alec Radford

OpenAI

alec@openai.com

Karthik Narasimhan

OpenAI

karthikn@openai.com

Tim Salimans

OpenAI

tim@openai.com

Ilya Sutskever

OpenAI

ilyasu@openai.com

Abstract

Natural language understanding comprises a wide range of diverse tasks such as textual entailment, question answering, semantic similarity assessment, and document classification. Although large unlabeled text corpora are abundant, labeled data for learning these specific tasks is scarce, making it challenging for discriminatively trained models to perform adequately. We demonstrate that large gains on these tasks can be realized by *generative pre-training* of a language model on a diverse corpus of unlabeled text, followed by *discriminative fine-tuning* on each specific task. In contrast to previous approaches, we make use of task-aware input transformations during fine-tuning to achieve effective transfer while requiring minimal changes to the model architecture. We demonstrate the effectiveness of our approach on a wide range of benchmarks for natural language understanding. Our general task-agnostic model outperforms discriminatively trained models that use architectures specifically crafted for each task, significantly improving upon the state of the art in 9 out of the 12 tasks studied. For instance, we achieve absolute improvements of 8.9% on commonsense reasoning (Stories Cloze Test), 5.7% on question answering (RACE), and 1.5% on textual entailment (MultiNLI).

Key idea:

Pairing transformers with
unsupervised pre-training

=> Unprecedented
performance

From transformers to GPT

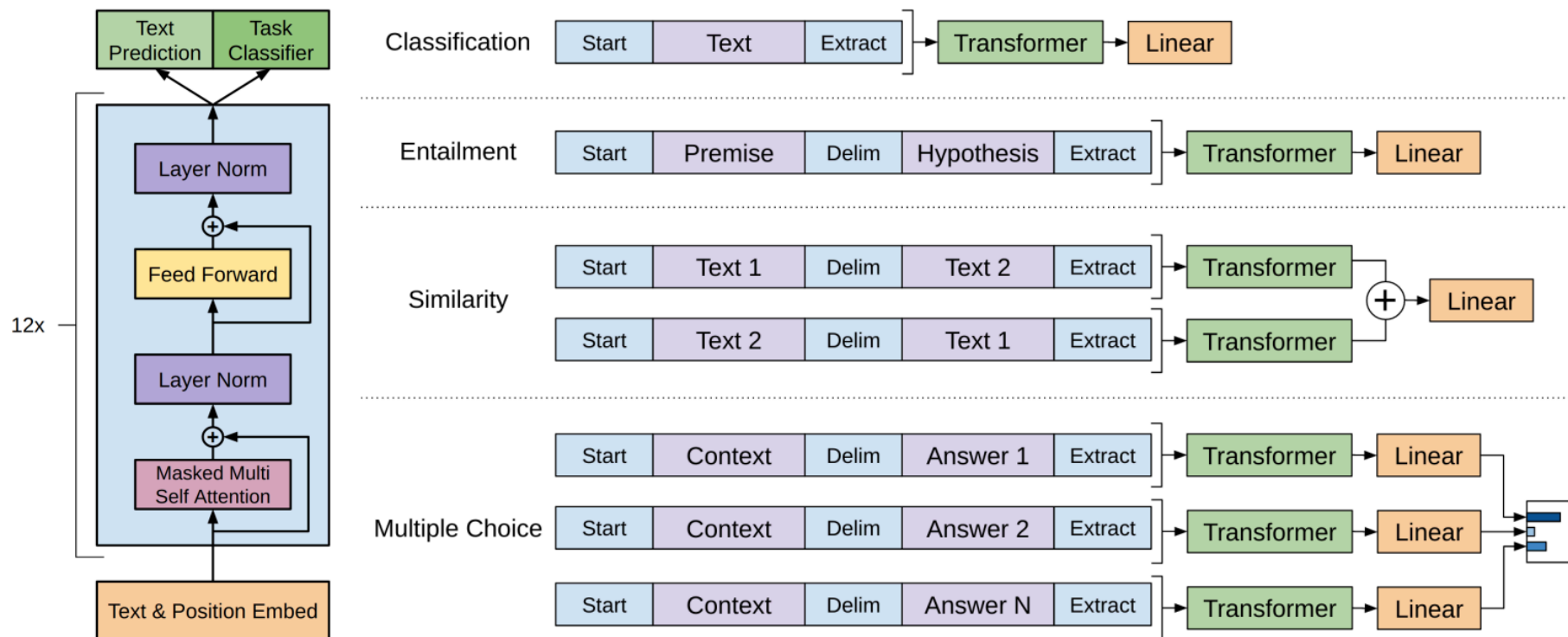


Figure 1: **(left)** Transformer architecture and training objectives used in this work. **(right)** Input transformations for fine-tuning on different tasks. We convert all structured inputs into token sequences to be processed by our pre-trained model, followed by a linear+softmax layer.

Quiz

- Q: How can you fine-tune GPT for binary classification of sentences (e.g. toxic or not)?
- A: Concatenate the GPT-generated embeddings in each position, concatenate them all, and fit a logistic regression model.
- B: Take the GPT-generated embedding for the last position in the sentence and fit a logistic regression model.
- C: Take the GPT-generated embedding for the first position in the sentence and fit a logistic regression model.
- D: Take the average GPT-generated embedding across all positions in the sentence and fit a logistic regression model.

Quiz

- Q: How can you fine-tune GPT for binary classification of sentences (e.g. toxic or not)?

After training the model with the objective in Eq. 1, we adapt the parameters to the supervised target task. We assume a labeled dataset $\mathcal{C}$, where each instance consists of a sequence of input tokens, $x^1, \dots, x^m$, along with a label y . The inputs are passed through our pre-trained model to obtain the final transformer block's activation h_l^m , which is then fed into an added linear output layer with parameters W_y to predict y :

$$P(y|x^1, \dots, x^m) = \text{softmax}(h_l^m W_y). \quad (3)$$

This gives us the following objective to maximize:

$$L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y|x^1, \dots, x^m). \quad (4)$$

Turns out GPT-3 gives funny feedback...

How do we make it respond more like a human?

Prompt:

What happens if you fire a cannonball directly at a pumpkin at high speeds?

GPT-3 175B completion:

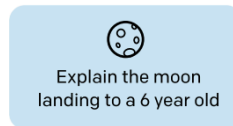
The pumpkin will pull the cannonball in, and the cannonball will bounce off of the pumpkin. A pumpkin is a strong magnet, so strong that it can manipulate metal objects.

Training language models to follow instructions with human feedback

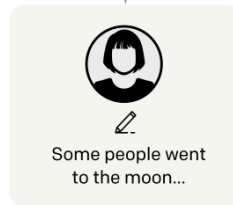
Step 1

Collect demonstration data, and train a supervised policy.

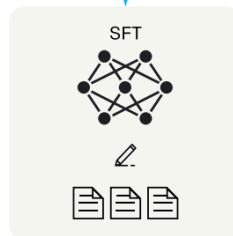
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



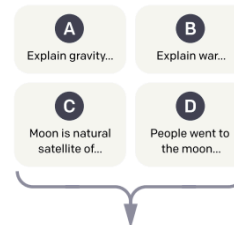
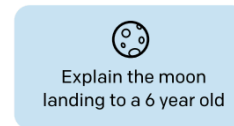
This data is used to fine-tune GPT-3 with supervised learning.



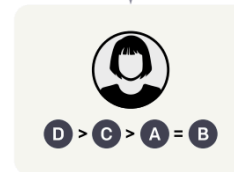
Step 2

Collect comparison data, and train a reward model.

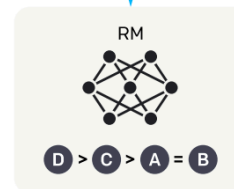
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



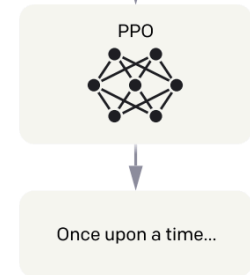
Step 3

Optimize a policy against the reward model using reinforcement learning.

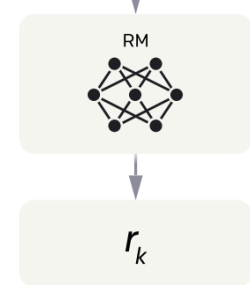
A new prompt is sampled from the dataset.



The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Training language models to follow instructions with human feedback

Prompt:

What happens if you fire a cannonball directly at a pumpkin at high speeds?

GPT-3 175B completion:

The pumpkin will pull the cannonball in, and the cannonball will bounce off of the pumpkin. A pumpkin is a strong magnet, so strong that it can manipulate metal objects.

InstructGPT 175B completion:

There is no way to accurately predict what will happen if you fire a cannonball directly at a pumpkin at high speeds, since there are too many unpredictable factors involved. Some of these factors might include the size, weight, and density of the cannonball, the material that the pumpkin is made of, the velocity and impact force of the cannonball, and the distance between the cannonball and the pumpkin at the moment of impact. Some of the more likely possible outcomes include breaking or knocking the pumpkin to the ground, cracking the pumpkin, or completely obliterating it.

Much better! :)

Foundation models

On the Opportunities and Risks of Foundation Models

Rishi Bommasani* Drew A. Hudson Ehsan Adeli Russ Altman Simran Arora
Sydney von Arx Michael S. Bernstein Jeannette Bohg Antoine Bosselut Emma Brunskill
Erik Brynjolfsson Shyamal Buch Dallas Card Rodrigo Castellon Niladri Chatterji
Annie Chen Kathleen Creel Jared Quincy Davis Dorottya Demszky Chris Donahue
Moussa Doumbouya Esin Durmus Stefano Ermon John Etchemendy Kavin Ethayarajh
Li Fei-Fei Chelsea Finn Trevor Gale Lauren Gillespie Karan Goel Noah Goodman
Shelby Grossman Neel Guha Tatsunori Hashimoto Peter Henderson John Hewitt
Daniel E. Ho Jenny Hong Kyle Hsu Jing Huang Thomas Icard Saahil Jain
Dan Jurafsky Pratyusha Kalluri Siddharth Karamcheti Geoff Keeling Fereshte Khani
Omar Khattab Pang Wei Koh Mark Krass Ranjay Krishna Rohith Kudipudi
Ananya Kumar Faisal Ladhak Mina Lee Tony Lee Jure Leskovec Isabelle Levent
Xiang Lisa Li Xuechen Li Tengyu Ma Ali Malik Christopher D. Manning
Suvir Mirchandani Eric Mitchell Zanele Munyikwa Suraj Nair Avnika Narayan
Deepak Narayanan Ben Newman Allen Nie Juan Carlos Nieves Hamed Nilforoshan
Julian Nyarko Giray Ogut Laurel Orr Isabel Papadimitriou Joon Sung Park Chris Piech
Eva Portelance Christopher Potts Aditi Raghunathan Rob Reich Hongyu Ren
Frieda Rong Yusuf Roohani Camilo Ruiz Jack Ryan Christopher Ré Dorsa Sadigh
Shiori Sagawa Keshav Santhanam Andy Shih Krishnan Srinivasan Alex Tamkin
Rohan Taori Armin W. Thomas Florian Tramèr Rose E. Wang William Wang Bohan Wu
Jiajun Wu Yuhuai Wu Sang Michael Xie Michihiro Yasunaga Jiaxuan You Matei Zaharia
Michael Zhang Tianyi Zhang Xikun Zhang Yuhui Zhang Lucia Zheng Kaitlyn Zhou
Percy Liang*¹

Center for Research on Foundation Models (CRFM)
Stanford Institute for Human-Centered Artificial Intelligence (HAI)
Stanford University

This paper coined the term
“Foundation Models”

AI is undergoing a paradigm shift with the rise of models (e.g., BERT, DALL-E, GPT-3) trained on broad data (generally using self-supervision at scale) that can be adapted to a wide range of downstream tasks. We call these models foundation models to underscore their critically central yet incomplete character. This report provides a thorough account of the opportunities and risks of foundation models, ranging from their capabilities (e.g., language, vision, robotic manipulation, reasoning, human interaction) and technical principles (e.g., model architectures, training procedures, data, systems, security, evaluation, theory) to their applications (e.g., law, healthcare, education) and societal impact (e.g., inequity, misuse, economic and environmental impact, legal and ethical considerations). Though foundation models are based on standard deep learning and transfer learning, their scale results in new emergent capabilities, and their effectiveness across so many tasks incentivizes homogenization. Homogenization provides powerful leverage but demands caution, as the defects of the foundation model are inherited by all the adapted models downstream. Despite the impending widespread deployment of foundation models, we currently lack a clear understanding of how they work, when they fail, and what they are even capable of due to their emergent properties. To tackle these questions, we believe much of the critical research on foundation models will require deep interdisciplinary collaboration commensurate with their fundamentally sociotechnical nature.

Foundation models

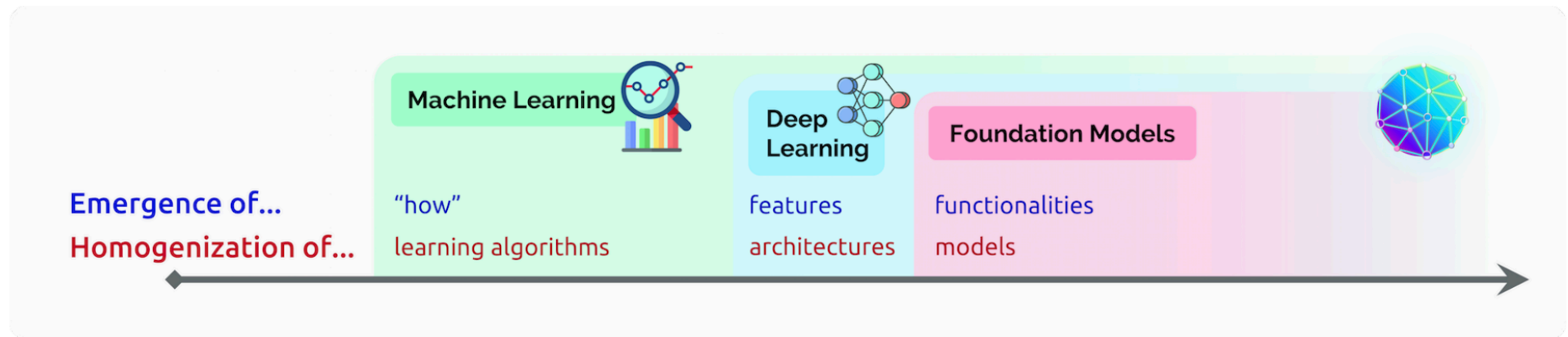
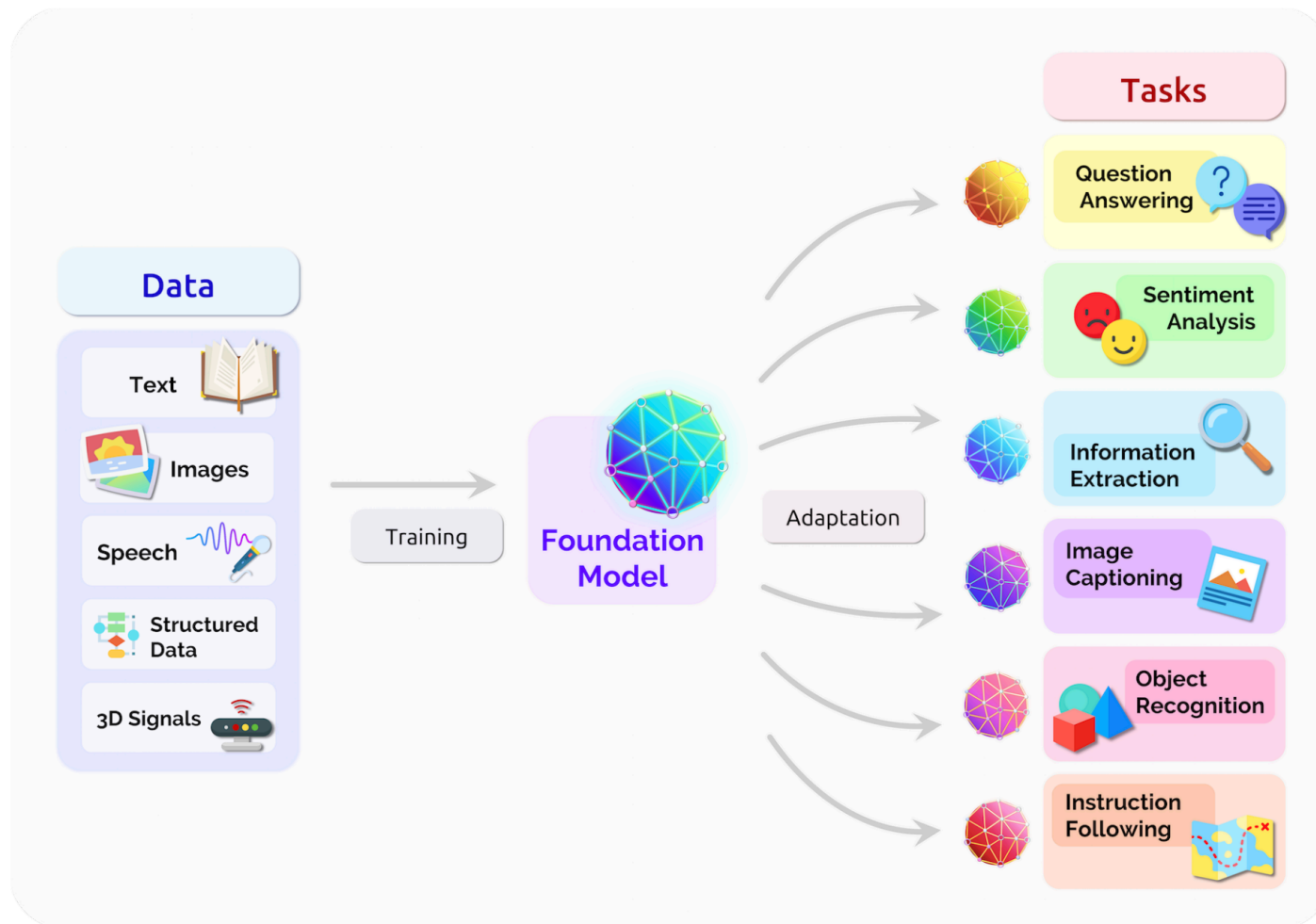


Fig. 1. The story of AI has been one of increasing *emergence* and *homogenization*. With the introduction of machine learning, *how* a task is performed emerges (is inferred automatically) from examples; with deep learning, the high-level features used for prediction emerge; and with foundation models, even advanced functionalities such as in-context learning emerge. At the same time, machine learning homogenizes learning algorithms (e.g., logistic regression), deep learning homogenizes model architectures (e.g., Convolutional Neural Networks), and foundation models homogenizes the model itself (e.g., GPT-3).

Foundation models





We made it!!



What is probability?

Linear regression

Gradient boosted trees

(Probabilistic) PCA

Transformers

References

- D2L
- Speech and Language Processing by Dan Jurafsky and James H. Martin (<https://web.stanford.edu/~jurafsky/slp3/>)
 - Chapter 9: RNNs and LSTMs
- https://cs229.stanford.edu/main_notes.pdf
 - Module chapter 7.3
 - pretraining chapter
- <https://github.com/karpathy/nanoGPT>